Claude editing setup pack

INSTRUCTIONS FOR CLAUDE CODE (read this page first)

This PDF is a channel-neutral AI video-editing toolkit: the pipeline contract (CLAUDE.md), six editing skills with their helper scripts, a planning skill, settings and a safety hook. It carries no brand, style, preferences, fonts, logos
or music. Those are supplied later, per channel or per video.

1. GET THE FILES FROM THE ZIP, NOT FROM THE PRINTED TEXT.

Text copied out of a PDF loses indentation, and Python scripts break without it. The exact files are in claude-editing-setup-pack.zip, which should sit next to this PDF; it is also attached inside this PDF. If only the PDF is here, pull the attachment out with:

```
uv run --with pypdf python -c "import pypdf;[open(n,'wb').write(d[0]) for n,d in pypdf.PdfReader('claude-editing-setup-pack.pdf').attachments.items()]"
```

The printed pages are the human-readable reference and the fallback if the zip is lost (then recreate each file at the path in its grey header bar, restoring indentation carefully, and keep characters exact; a few symbols such as arrows and emoji are printed as plain-text stand-ins).

2. PICK THE PROJECT FOLDER. Ask the user where the editing folder should live. Unzip so that the contents of the zip's video-editor/ folder land in it: CLAUDE.md at its root, skills in .claude/skills/<name>/. Keep scripts executable (chmod +x .claude/skills/rough-cut/scripts/transcribe.sh). Then create the empty folders projects/, styles/ and assets/ (with assets/fonts, assets/logos, assets/music, assets/sfx, assets/luts inside). Jobs go in projects/, one kebab-case folder per video.

3. BRAND AND STYLE. CLAUDE.md and the skills expect brand.md at the root and styles/<style>/style.md + style.json; the scripts stop without brand.md. Ask the user: do they already have a brand and style to feed in, or should you build them from zero? If they have them, write them to those paths. If not, run the interview in setup/brand-and-style-interview.md, using setup/brand.template.md and setup/style.template.json (they carry the exact field names the scripts read). Never invent brand choices without asking. Delete setup/ when brand.md and the first style are done.

4. PLAN FIRST, ALWAYS. CLAUDE.md makes the edit-plan skill a standing rule: before any editing stage, and before acting on any notes, show numbered steps with a time estimate for each and a total. Tell the user this is how every edit will start. The premiere-xml-cut skill has no shared scripts on purpose; it says which scripts to write inside each job.

5. INSTALL THE STACK (next page), then run the "Verify the stack" block from CLAUDE.md and report what fails.

6. NOT INCLUDED ON PURPOSE: a filled-in brand.md or style, fonts, logos, LUTs, music, sound effects, editing-preference memories, .claude/settings.local.json, old projects.

7. When everything checks out, tell the user in plain words what works, and list anything they must do themselves (installs that need their password, brand/style files to provide).

Install checklist

INSTALL CHECKLIST (macOS, Homebrew)

Command line tools
- Homebrew, then: brew install ffmpeg uv node git
- Node must be version 22 or newer (node --version).
- Python 3 with Pillow: python3 -m pip install pillow (or uv pip). numpy is used by several scripts too.
- HyperFrames (motion graphics), pinned: npm install in the project root (package.json pins hyperframes 0.8.73). Never upgrade it silently. Check: npx hyperframes@0.8.73 doctor (an upgrade nag and "Docker not running" are expected and fine).
- WhisperX (word-level transcripts): .claude/skills/rough-cut/scripts/transcribe.sh runs it through uv; the first run downloads the model. Read the rough-cut SKILL.md for the exact call.

Claude Code plugin
- The "watch" plugin (skill watch:watch): lets Claude see renders by extracting frames. Install it from the Claude Code plugin marketplace, then add GROQ_API_KEY or OPENAI_API_KEY to ~/.config/watch/.env if you want its Whisper fallback (optional).
- Connectors are not part of this setup yet. Where a skill mentions one (Higgsfield in broll and graphics), skip that route and ask the user.

What each skill does (pipeline order, from CLAUDE.md)
- edit-plan runs first every time: numbered steps, a time estimate each, a total 1 rough-cut WhisperX transcript -> cutsheet.json -> one FFmpeg splice 1 premiere-xml-cut same stage, but hands Premiere Pro an FCP7 XML timeline instead of a render 2 graphics HyperFrames (HTML/CSS/GSAP) cards and callouts composited with FFmpeg 3 broll HyperFrames motion graphics for beats with no footage 4 finishing captions, music bed, sound effects 5 export promote the final render, tidy the job folder hook guard-delete blocks risky delete commands before they run

Style is chosen per video from styles/ and brand.md holds colour tokens and fonts, both supplied later.
Always refer to colours as tokens ($accent, $bg), never hex.

Contents

PROJECT ROOT, SETTINGS, HOOK

CLAUDE.md (5,041 bytes)
package.json (78 bytes)
.claude/settings.json (1,874 bytes)
.claude/hooks/guard-delete.py (3,900 bytes)

BRAND AND STYLE INTERVIEW

setup/brand-and-style-interview.md (5,644 bytes)
setup/brand.template.md (2,596 bytes)
setup/style.template.json (2,603 bytes)

SKILL: EDIT-PLAN

.claude/skills/edit-plan/SKILL.md (5,471 bytes)

SKILL: ROUGH-CUT

.claude/skills/rough-cut/SKILL.md (11,897 bytes)
.claude/skills/rough-cut/scripts/beat_snap.py (3,866 bytes)
.claude/skills/rough-cut/scripts/montage.py (9,552 bytes)
.claude/skills/rough-cut/scripts/motion_check.py (3,360 bytes)
.claude/skills/rough-cut/scripts/remap.py (3,281 bytes)
.claude/skills/rough-cut/scripts/sound_scan.py (1,965 bytes)
.claude/skills/rough-cut/scripts/splice.py (21,772 bytes)
.claude/skills/rough-cut/scripts/strip.py (2,048 bytes)
.claude/skills/rough-cut/scripts/transcribe.sh (2,110 bytes)
.claude/skills/rough-cut/scripts/unknown_words.py (2,672 bytes)
.claude/skills/rough-cut/scripts/words.py (1,278 bytes)

SKILL: GRAPHICS

.claude/skills/graphics/SKILL.md (13,463 bytes)
.claude/skills/graphics/scripts/dupe_check.py (2,361 bytes)
.claude/skills/graphics/scripts/validate_plan.py (2,579 bytes)

SKILL: BROLL

.claude/skills/broll/SKILL.md (4,201 bytes)
.claude/skills/broll/scripts/finalize.py (3,097 bytes)

SKILL: FINISHING

.claude/skills/finishing/SKILL.md (9,321 bytes)
.claude/skills/finishing/scripts/beat_grid.py (4,443 bytes)
.claude/skills/finishing/scripts/captions.py (17,950 bytes)
.claude/skills/finishing/scripts/find_release.py (2,272 bytes)
.claude/skills/finishing/scripts/mix.py (12,182 bytes)
.claude/skills/finishing/scripts/track_profile.py (2,048 bytes)

SKILL: EXPORT

.claude/skills/export/SKILL.md (2,435 bytes)
.claude/skills/export/scripts/export.py (5,093 bytes)

SKILL: PREMIERE-XML-CUT

.claude/skills/premiere-xml-cut/SKILL.md (9,123 bytes)

Project root, settings, hook

FILE: CLAUDE.md

5,041 bytes

````
# Video editor: the pipeline contract

This folder is an AI video editor. Claude cannot open Premiere, so every stage is
built from things Claude can actually do: read a transcript, make a decision, write
code, and run a command line tool.

## Two halves

- **Taste, permanent, shared by every video:** `brand.md`, `styles/`, `assets/`.
- **Jobs, disposable:** `projects/<job>/`.

Anything that does not clearly belong to one of those halves means the design has
drifted. Do not add notes files, config files or "temp" folders.

## The pipeline

| # | Stage     | Skill        | Runs on                 |
|---|-----------|--------------|-------------------------|
| 0 | Setup     | (done once)  | Claude                  |
| 1 | Rough cut | `rough-cut`  | WhisperX + FFmpeg       |
| 2 | Graphics  | `graphics`   | HyperFrames             |
| 3 | B-roll    | `broll`      | HyperFrames + Higgsfield|
| 4 | Finishing | `finishing`  | FFmpeg                  |
| 5 | Export    | `export`     | FFmpeg                  |

Short form and long form run the same five stages. Only how graphics and captions
behave inside stages 2 and 4 changes, and that is driven entirely by the style file.
Every stage renders, then gets reviewed with the `watch` skill in sub-agents
(see "Review loop" below) before the next stage starts.

## Job layout

```
projects/<job>/            kebab-case, named after the content
|--- raw/                   source clips, copied in, never moved
|--- broll/                 screen recordings and supporting footage
|--- audio/                 licensed music, sound samples for this video
|--- assets/                references, screenshots, logos for this video
|--- transcript/            transcript.json and cutsheet.json, the durable record
|--- graphics-build/        the composition source. This is the real progress
`--- outputs/               renders and the cut-aligned transcript
```

Job naming is a rule: name the folder after what the video is about, in kebab case.
Never the camera filename, never a date, never a stage suffix like `-final` or `-v2`.
One folder carries one content piece through every stage.

## Standing rules

- **Plan first, every time.** Before starting any editing job or stage, and before acting on
  any notes or corrections, use the `edit-plan` skill: show numbered steps with a time
  estimate for each and a total, then start. Report actual time against it as you go.
- **Plan first, every time.** Before starting any editing job or stage, and before acting on
  any notes or corrections, use the `edit-plan` skill: show numbered steps with a time
  estimate for each and a total, then start. Report actual time against it as you go.
- Read `brand.md` and the chosen `styles/<style>/style.md` + `style.json` before any
  creative decision. Colours are always referenced as tokens (`$accent`, `$bg`), never hex.
- Every rule lives in the skill that owns it. Read the skill before running its stage.
- Local direction (script comments, one-off asks) never silently overrides a style
  convention. Flag the conflict and ask.
- Any correction in final review that should apply to every future video gets written
  back into the style file (show the user the diff first) before the job is closed.
  One-off notes are applied and forgotten.
- Keep build source durable, never only in a temp folder.
- Never prefix commands with cd; you're already in the project folder. Use relative paths, run one
  command at a time (no &&, ; or pipes unless necessary).
- Helper scripts go in the current project's tmp/ folder (projects/<job>/tmp/), never in
  $CLAUDE_JOB_DIR, /tmp or anywhere else. Run them as `python3 tmp/<name>.py` from the project folder.

## Review loop

Claude cannot see video; the `watch` skill extracts frames so it can. After every render:

1. Send the review to **sub-agents** (frame dumps flood the main context). They return
   timestamped findings.
2. Pass one, **technical QA** as a checklist: stretched assets, vanished elements, wrong
   colours, brightness dips at seams, safe-zone violations, bad tail frames.
````

````
3. Pass two, **composition** as its own named step: every overlay re-checked against what
   the style file says for that element category; every distinct visual moment in the
   direction accounted for as built or explicitly flagged as skipped, never silently
   simplified away.
4. Fix, re-render, review again until it passes.
5. Also run an early spot review after the first ~10% of a graphics build.

Use frame extraction, not a video-understanding model: Claude decides exactly where to look.

## Verify the stack

Before the first job, and whenever something "doesn't work":

```bash
ffmpeg -version && ffprobe -version
node --version        # must be >= 22
uv --version
python3 -c "import PIL"
npx hyperframes@<pinned> doctor   # upgrade nag and Docker-not-running are expected
```

Then confirm the `watch` skill appears in Claude's tool list.
HyperFrames is pinned per workspace; never upgrade it silently.
````

FILE: package.json

78 bytes

```
{
  "private": true,
  "devDependencies": {
    "hyperframes": "0.8.73"
  }
}
```

FILE: .claude/settings.json

1,874 bytes

```
{
  "permissions": {
    "defaultMode": "acceptEdits",
     "allow": [
      "Read(./**)",
      "Bash(ffmpeg *)",
      "Bash(ffprobe *)",
      "Bash(python3 .claude/skills/*)",
      "Bash(python3 projects/*)",
      "Bash(python3 tmp/*)",
      "Bash(projects/*/tmp/venv/bin/python .claude/skills/*)",
      "Bash(projects/*/tmp/venv/bin/python projects/*)",
      "Bash(ls *)",
      "Bash(grep *)",
      "Bash(head *)",
      "Bash(find *)",
      "Bash(git status *)",
      "Bash(git diff *)",
      "WebFetch(domain:www.googleapis.com)",
      "WebFetch(domain:youtube.googleapis.com)",
      "Bash(cp *)",
      "Bash(mkdir *)",
      "Bash(git add *)",
      "Bash(git commit *)"
    ],
    "ask": [
      "Bash(git push *)",
      "Bash(brew install *)",
      "Bash(brew upgrade *)",
      "Bash(brew reinstall *)",
      "Bash(npm install *)",
      "Bash(npm i *)",
      "Bash(npm ci *)",
      "Bash(pnpm add *)",
      "Bash(yarn add *)",
      "Bash(pip install *)",
      "Bash(pip3 install *)",
      "Bash(python3 -m pip *)",
      "Bash(uv add *)",
      "Bash(uv sync *)",
      "Bash(uv pip install *)",
      "Bash(uv tool install *)",
      "Bash(uv run --with *)",
      "Bash(gem install *)",
```

```
      "Bash(cargo install *)"
        ],
    "deny": [
      "Bash(git push*)",
      "Bash(git reset --hard*)",
      "Bash(git clean*)"
    ]
  },
  "sandbox": {
    "network": {
      "allowedDomains": [
        "www.googleapis.com",
        "youtube.googleapis.com",
        "youtubeanalytics.googleapis.com",
        "oauth2.googleapis.com"
      ]
    }
  },
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "Bash",
        "hooks": [
          {
            "type": "command",
            "command": "python3 \"$CLAUDE_PROJECT_DIR/.claude/hooks/guard-delete.py\"",
            "timeout": 10
          }
        ]
      }
    ]
  }
}
```

FILE: .claude/hooks/guard-delete.py

3,900 bytes

```
#!/usr/bin/env python3
"""PreToolUse hook: prompt before any Bash delete outside thumbnails/ or projects/*/tmp.

Silent (no output) when a command deletes nothing, or only deletes inside those
two places. Otherwise it returns permissionDecision "ask" so Claude Code prompts,
even with sandbox auto-allow or acceptEdits on.
"""
import json
import os
import re
import shlex
import sys

SEPARATORS = {";", "&&", "||", "|", "&", "(", ")", "\n"}
DELETERS = {"rm", "rmdir", "unlink", "srm", "trash"}
WRAPPERS = {"sudo", "command", "builtin", "nice", "nohup", "time", "env"}


def ask(reason):
    print(json.dumps({"hookSpecificOutput": {
        "hookEventName": "PreToolUse",
        "permissionDecision": "ask",
        "permissionDecisionReason": reason,
    }}))
    sys.exit(0)


def safe_target(raw, cwd, project):
    if any(c in raw for c in "$`") or raw.startswith("~"):
        return False
    path = os.path.normpath(os.path.join(cwd, raw))
    # Resolve symlinked parent dirs; the final component is what rm removes.
    path = os.path.join(os.path.realpath(os.path.dirname(path)), os.path.basename(path))
    rel = os.path.relpath(path, project)
    parts = rel.split(os.sep)
    if parts[0] == "..":
        return False
    if parts[0] == "thumbnails" and len(parts) >= 2:
        return True
    if len(parts) >= 3 and parts[0] == "projects" and parts[2] == "tmp":
        return True
    return False


def segments(tokens):
    seg = []
```

```
    for t in tokens:
        if t in SEPARATORS:
            if seg:
                yield seg
            seg = []
        else:
            seg.append(t)
    if seg:
        yield seg


def main():
    data = json.load(sys.stdin)
    cmd = (data.get("tool_input") or {}).get("command", "")
    cwd = data.get("cwd") or os.getcwd()
    project = os.path.realpath(os.environ.get("CLAUDE_PROJECT_DIR") or cwd)

    try:
        lexer = shlex.shlex(cmd, posix=True, punctuation_chars=";&|()")
        lexer.whitespace_split = True
        tokens = list(lexer)
    except ValueError:
        if re.search(r"\b(rm|rmdir|unlink|-delete)\b", cmd):
            ask("Could not parse a command that looks like it deletes files.")
        return

    for seg in segments(tokens):
        while seg and (seg[0] in WRAPPERS or re.match(r"^\w+=", seg[0])):
            seg = seg[1:]
        if not seg:
            continue
        prog = os.path.basename(seg[0])
        args = seg[1:]

        if prog in ("bash", "sh", "zsh", "eval", "xargs", "parallel"):
            if any(re.search(r"\b(rm|rmdir|unlink)\b|-delete\b", a) for a in args):
                ask(f"`{prog}` runs a delete whose targets can't be checked.")
            continue

        if prog == "git" and args[:1] and args[0] in ("rm", "clean"):
            ask(f"`git {args[0]}` deletes files.")

        targets = None
        if prog in DELETERS:
            targets, after_dd = [], False
            for a in args:
                if a == "--" and not after_dd:
                    after_dd = True
                elif after_dd or not a.startswith("-"):
                    targets.append(a)
        elif prog == "find" and ("-delete" in args or any(
                a in ("-exec", "-execdir", "-ok", "-okdir") and i + 1 < len(args)
                and os.path.basename(args[i + 1]) in DELETERS
                for i, a in enumerate(args))):
            targets = []
            for a in args:
                if a.startswith("-") or a in ("(", "!"):
                    break
                targets.append(a)
            targets = targets or ["."]

        if targets is None:
            continue
        if not targets:
            ask(f"`{prog}` with no visible targets.")
        bad = [t for t in targets if not safe_target(t, cwd, project)]
        if bad:
            ask("Deletes outside thumbnails/ and projects/*/tmp: " + ", ".join(bad))


if __name__ == "__main__":
    main()
```

Brand and style interview

FILE: setup/brand-and-style-interview.md

5,644 bytes

```
# Brand and style interview

Run this once per channel, after the skills are installed and before the first edit.
It produces `brand.md` at the project root and one starter style in `styles/<style>/`.
The skill scripts read both files, so the field names below are not optional.

Ask one topic at a time, in plain words, and wait for the answer. Offer two or three
concrete suggestions when the user is unsure, instead of leaving a field blank. Nothing in
brand.md may stay empty or say "TBD": if the user truly has no answer, pick a sensible
default, say so, and mark it in the prose as a default they can change later.

When done, show the user both files and the folder tree, then delete this `setup/` folder.

---

## Part 1: brand.md

Start from `setup/brand.template.md`. Fill it in from these questions.

1. **The channel.** What is it called, who is on camera, and what is it about in one or
   two sentences? Who watches it, and what do they already know (so captions and graphics
   never over- or under-explain)?
2. **The feel.** Three or four words for how everything on screen should feel. Ask for a
   channel or video they love the look of, if they have one.
3. **Colours.** Six tokens, each with one role and one hex value:
   - `bg` background base, also light text over footage if that suits the brand
   - `ink` main text colour on light surfaces
   - `accent` the one loud colour, used sparingly (one element per scene)
   - `accent-soft` a quieter tint of the accent for soft highlights
   - `rule` hairlines and dividers
   - `muted` secondary labels
   Add `shadow` (usually `#000000`) if text will sit over footage. Extra brand colours are
   fine as extra tokens, but keep the core six. If they have a logo, pull candidate colours
   from it and confirm. Check `bg`/`ink` contrast is readable.
4. **Logo.** Ask for logo files (PNG with transparency, a light and a dark version if they
   exist). Save them to `assets/logos/` and list which goes on light vs dark.
5. **Fonts.** One display font (titles, cards) and one caption font; they may be the same.
   The actual font files must go in `assets/fonts/` because renders cannot use system
   fonts. Prefer open-licence fonts (Google Fonts, SIL OFL) and save the licence file next
   to the font. Ask: all caps or mixed case?
6. **Caption voice.** Exact words or lightly cleaned? Spelling (US/UK)? Emoji allowed in
   captions (default no: emoji hang the graphics renderer)? Any emphasis colour?
7. **Hook.** How should a video open: straight on the strongest line, a cold open, a hook
   card? Is there a default hook text, or none?
8. **Speaker.** Name and role as they should appear on screen; whether lower thirds are used.
9. **Mishear list.** Names, places, products and jargon the transcriber will get wrong:
   the user's own name, the channel name, brand and product names. Write each as heard ->
   correct. Only single-word, whole-word swaps go in the machine block (multi-word fixes
   would break word timings); note multi-word ones in the prose for manual fixing.

Keep the colours in two places: the readable table and the JSON block at the bottom.
They must match. Styles then refer to colours only as tokens (`$accent`), never hex.

---

## Part 2: the starter style

Start from `setup/style.template.json` and write `styles/<name>/style.json` plus
`styles/<name>/style.md`. Name the style after the look, in kebab case (e.g. `talking-head`,
`explainer-shorts`).

The bar for style.md: someone should be able to edit a new video in this style from the
file alone, without seeing the reference. Adjectives are not enough.

1. **Formats.** Which of `long` (16:9), `short-explainer` (9:16, graphics top, face
   bottom) and `short-raw` (9:16, face full frame, one hook card) is this style for?
   One style can cover several.
2. **Reference.** Is there a reference video (their own or one they admire)? If yes, study
   it in two passes with the watch skill:
   - Pass one over the whole runtime at roughly a hundred scene-change frames: what scene
     types appear, what they are called, how often they alternate, typical shot length.
   - Pass two at a high frame rate on three to five boundaries (face into full-screen
     graphic, graphic back to face, graphic into graphic, anything odd from pass one):
     how the joins actually work.
   Take the mechanics, pacing and placement; map them onto this brand's colours and fonts.
   Never copy another channel's assets. If the reference is the user's own channel, its
   colours are the brand and can be used as they are.
```

```
   If there is no reference, ask the questions below directly.
3. **Pacing.** How tight is the cut: keep short pauses or cut every gap? Longest pause
   to keep, in seconds.
4. **Graphics.** Which kinds are allowed (stat, card, screenshot, takeover, zoom, diagram,
   b-roll slot), how often, and where on screen. Transitions: hard cuts or animated?
   Zooms/punch-ins on the face: yes or no?
5. **Captions.** On or off per format (long form usually off). Words per line, size,
   position, case, colour tokens. Highlight: `karaoke` (each word lights up as spoken),
   `keywords` (only key words coloured) or `none`.
6. **Audio.** Music under voice: yes/no, and how loud. Sound effects: sparse, a handful per
   video, real samples only. Natural sound kept or muted under voice.
7. **Ending.** End screen space, outro card, or none.

Write every decision as a value in style.json and as a sentence (with the reason) in
style.md. Leave `learned` as an empty list: corrections from future reviews that should
apply to every video get added there, after showing the user the diff.
```

FILE: setup/brand.template.md

2,596 bytes

```
# Brand: <channel name>

<Who is on camera and what the channel is about, in two or three sentences.>

**Audience:** <who watches, what they already know, what never to assume.>

Everything on screen should feel **<three or four words>**.

## Logo

Files in `assets/logos/`.

| File | Use on |
|------|--------|
| `<logo-dark>.png` | light backgrounds (default) |
| `<logo-light>.png` | dark backgrounds and over footage |

## Colours

Six core tokens. Styles refer to these as `$token`, never as hex.

| Token        | Role                                    | Hex       |
|--------------|-----------------------------------------|-----------|
| bg           | <background base>                       | `#......` |
| ink          | <main text on light surfaces>           | `#......` |
| accent       | <the one loud colour, used sparingly>   | `#......` |
| accent-soft  | <quieter tint of the accent>            | `#......` |
| rule         | <hairlines and dividers>                | `#......` |
| muted        | <secondary labels>                      | `#......` |
| shadow       | <drop shadow behind text over footage>  | `#000000` |

## Type

- **Display font:** <name> (`assets/fonts/<file>`), for titles and cards. Licence: <licence file>.
- **Caption font:** <name> (`assets/fonts/<file>`).
- <case: all caps or mixed; weights used.>

## Caption voice

<Exact words or lightly cleaned; spelling; emoji rule; emphasis rule.>

## Hook rule

<How every video opens; default hook text or none.>

## Mishear list

Single-word, whole-word swaps only (they are applied automatically to every transcript).

| Heard | Correct |
|-------|---------|
| <heard> | <correct> |

<Multi-word fixes to make by hand, if any.>

## Speaker

- **Name:** <name>
- **Role:** <role>
- <lower thirds: used or not.>

## Machine-readable
```

Skills parse this block. Keep it in sync with the tables above.

```json
{
  "name": "<channel name>",
  "logo": "assets/logos/<logo-dark>.png",
  "logos": {
    "dark": "assets/logos/<logo-dark>.png",
    "light": "assets/logos/<logo-light>.png"
  },
  "colors": {
    "bg": "#FFFFFF",
    "ink": "#111111",
    "accent": "#FF5500",
    "accent-soft": "#FFD9C7",
    "rule": "#E5E5E5",
    "muted": "#6B6B6B",
    "shadow": "#000000"
  },
  "fonts": {
    "display": "<display font name>",
    "caption": "<caption font name>"
  },
  "font_files": {
    "display": ["assets/fonts/<display file>"],
    "caption": ["assets/fonts/<caption file>"]
  },
  "speaker": {"name": "<name>", "role": "<role>"},
  "hook_default": null,
  "mishears": {"<heard lowercase>": "<Correct>"}
}
```

FILE: setup/style.template.json

2,603 bytes

```
{
  "name": "<style-name>",
  "status": "from-interview",
  "formats": ["long", "short-explainer", "short-raw"],
  "fps": "source",

  "fonts": {
    "display": "$display",
    "caption": "$caption",
    "case": "mixed"
  },

  "rough_cut": {
    "hook": "open on the strongest line; no preamble before the hook",
    "pauses": {
      "long": {"keep_up_to_s": 0.4, "cut_over_s": 0.5, "cut_to_s": [0.2, 0.3]},
      "short-explainer": {"keep_up_to_s": 0.25, "cut_over_s": 0.3, "cut_to_s": [0.12, 0.15]},
      "short-raw": {"keep_up_to_s": 0.25, "cut_over_s": 0.3, "cut_to_s": [0.12, 0.15]}
    },
    "reframe": {
      "short-raw": {"canvas": [1080, 1920], "picture_height": 1920, "bars": "none",
                    "x_offset": "per segment in cutsheet, px, positive = picture moves right"}
    }
  },

  "captions": {
    "long": {"enabled": false},
    "short-explainer": {
      "enabled": true, "words_per_line": [2, 4], "y": 1500, "size_px": 90, "min_size_px": 72,
      "color": "$bg", "active_word_color": "$accent", "highlight": "karaoke",
      "stroke_color": "$shadow", "stroke_px": 0, "shadow_px": 3, "shadow_color": "$shadow",
      "shadow_opacity": 0.45, "uppercase": false, "strip_punctuation": true
    },
    "short-raw": {
      "enabled": true, "words_per_line": [2, 4], "y": 1500, "size_px": 96, "min_size_px": 80,
      "color": "$bg", "active_word_color": "$bg", "highlight": "none",
      "stroke_color": "$shadow", "stroke_px": 0, "shadow_px": 3, "shadow_color": "$shadow",
      "shadow_opacity": 0.45, "uppercase": false, "strip_punctuation": true
    }
  },

  "hook": {
    "card": "short-raw only: one hook card in the first seconds",
    "open": "strongest line"
  },

  "graphics": {
```

```
    "kinds": ["stat", "card", "screenshot", "takeover", "zoom", "diagram", "broll"],
    "frequency": "only lines that earn one: the hook, concrete stats or screenshots, payoffs",
    "transitions": "hard cuts between graphics; no full-frame/PiP bouncing",
    "zooms": true,
    "tail_margin_s": 0.5,
    "safe_zone": {
      "long": {"margin_pct": 10, "outro_keep_clear": "right 40% for end screen cards"},
      "short-explainer": {"x": [80, 1000], "y": [200, 1620]},
      "short-raw": {"x": [80, 1000], "y": [200, 1620]}
    }
  },

  "audio": {
    "music_bed_db": -18,
    "music_ducking": false,
    "sfx_db": -10,
    "sfx_per_video": "a handful, real samples only",
    "natural_sound_under_voice_db": -18
  },

  "outro": {
    "long": "end screen: keep the right 40% clear for the last 20 s",
    "short-explainer": "none",
    "short-raw": "none"
  },

  "learned": []
}
```

Skill: edit-plan

FILE: .claude/skills/edit-plan/SKILL.md

5,471 bytes

```
---
name: edit-plan
description: Lay out a numbered plan with a time estimate for every step before any editing work starts. Use at the start of every editing job or stage (rough cut, graphics, B-roll, finishing, export, Premiere XML), and every time the user gives notes, corrections or feedback to act on, before touching anything.
---

# Edit plan

Its whole job: before any editing work, show the user what will happen, in what order, and
how long each step will take, so they know what to expect and can change the plan before
time is spent. This is a standing rule, not an option: no stage starts and no batch of notes
is acted on without a plan first.

## When

- The start of a new video or a new stage of one.
- Every time the user sends notes, corrections or a list of changes.
- When the plan changes mid-way (a render fails, a note turns out bigger than it looked):
  show the revised plan before carrying on.

A one-line answer or a question about the edit needs no plan.

## Video or Premiere XML?

At the start of a new video, work out which output the user wants:

- **A rendered video** (`rough-cut` and the later stages): they asked to edit, cut, rough-cut
  or make a short, and said nothing about Premiere.
- **A Premiere Pro XML** (`premiere-xml-cut`): they said Premiere, XML or timeline.
- **Both**: they asked for both.

If the request does not say, do not assume. Ask as the first line of the plan, "A finished
video, a Premiere XML, or both?", and wait for the answer before step 1. This is the one case
where the plan waits. Once a job's output is settled, later rounds of notes on that job need
not ask again.

## Measure before estimating

Estimates come from the actual inputs, never from a guess about them. First run:

- `ffprobe` on the footage: number of clips, total minutes, resolution, frame rate.
- How long the finished video is meant to be (from the style or the user).
- Which stages this job needs, from the request and the style.
- What already exists in the job folder (a saved transcript means no transcription step).
- The machine: `sysctl -n machdep.cpu.brand_string` (macOS) or `nproc` (Linux), and whether
  a hardware video encoder works here.

## Starting estimates

Rough times on a recent Apple Silicon Mac. Scale them by the measured inputs and replace
them with the times this machine actually takes (see "Learn from real times").

| Step                                          | Starting estimate                                          |
|-----------------------------------------------|------------------------------------------------------------|
| First-ever setup of the transcription tool    | 10-20 min, once (3-5 GB download)                          |
| Transcription (WhisperX, CPU)                 | 0.5-1.5 x the footage length                               |
| Reading the transcript, writing the cut sheet | 5-15 min per 10 min of footage                             |
| Rough-cut render, hardware encoder            | ~15 s per minute of output (1080p)                         |
| Rough-cut render, software encoder (4K)       | 1.5-2.5 min per minute of output                           |
| Graphics plan                                 | 10-20 min                                                  |
| Each graphic: build, render, check            | 5-15 min                                                   |
| Composite graphics over the cut               | 1-3 min per minute of video                                |
| Generated B-roll clip                         | 3-10 min each                                              |
| Captions / music bed / sound effects          | 2-5 / 2-5 / 5-10 min                                       |
| One review pass (sub-agents, frame checks)    | 5-10 min                                                   |
| Fix round after a review                      | 5-15 min per group of related notes, plus re-render        |
| Export and tidy                               | 1-3 min                                                    |

Always add the review and at least one fix round to any step that renders.

## The plan the user sees

Short and plain. One line per step, in order:

```
Plan for <job> - <what this round does>
```
```

````
1. <step>  -  <what happens, in a few words>  -  ~<time>
2. ...

Total: about <time>. <What the user has to do, and when: e.g. "I'll need your answer on the
story after step 3 (about 25 min in)".>
Assumes: <the measured facts the estimates rest on: 42 min of 4K footage across 3 cameras, etc.>
```

- Give ranges when a step is uncertain ("10-20 min"), single numbers when it is not.
- Say which steps wait on the user, and roughly when they will be asked.
- For notes: one line per note (or per group of notes that touch the same thing), saying
  what will change and which part re-renders. Flag any note that conflicts with the style.
- Then start on step 1 (unless the output question above is still open). The user can stop
  or change the plan at any point.

## While working

- At the end of each step, one line: done, actual time vs estimate, what's next.
- If a step will overrun its estimate by more than about half, say so with the new estimate
  before carrying on, not after.

## Learn from real times

At the end of the job, compare actual times with the estimates. Where this machine is
consistently faster or slower (more than about half off, on two jobs or more), update the
"Starting estimates" table in this skill so the next plan is closer. Show the user the
change in one line.
````
````

Skill: rough-cut

FILE: .claude/skills/rough-cut/SKILL.md

11,897 bytes

````
---
name: rough-cut
description: Stage 1. Turn raw talking-head clips in projects/<job>/raw/ into the shortest cut that still delivers
» the value. Transcribes once with WhisperX (word-level timings), writes transcript/cutsheet.json, splices with one F
» Fmpeg filtergraph, and writes a transcript remapped onto the edited timeline. Use when the user drops footage and a
» sks to edit, cut, or rough-cut a video.
---

# Rough cut

Its whole job: turn raw talking-head clips into the shortest cut that still delivers the value.

Claude cannot hear audio. It only knows where a cut should land because WhisperX gives the
sub-second timing of every word, with forced alignment tight enough that cuts land on the
breath instead of near it.

## Inputs and outputs

| Reads                                  | Writes                                          |
|----------------------------------------|-------------------------------------------------|
| `projects/<job>/raw/*`                 | `transcript/transcript.json` (once, ever)       |
| `projects/<job>/broll/*`               | `transcript/broll.json` (B-roll narration)      |
| `brand.md` mishear list                | `transcript/cutsheet.json`                      |
| `transcript/mishears.json` (per video) | `outputs/base.mp4`                              |
|                                        | `outputs/transcript.cut.json` (remapped)        |

## Steps

1. **Transcribe once.** WhisperX `large-v3` with wav2vec2 alignment:
   `scripts/transcribe.sh projects/<job>`. Result goes to `transcript/transcript.json`.
   Re-running skips straight to the saved copy. This is the slowest step in the pipeline
   and runs exactly once per video.
2. **Also transcribe `broll/`** (same script does it, into `transcript/broll.json`).
   Creators narrate direction inside B-roll takes ("zoom in here", "use this for the
   pricing bit") that never appears in the main script.
3. **Mishear pass.** Run `scripts/unknown_words.py projects/<job>`. It prints only words
   that are neither ordinary English nor already known. Judge each in context: a recurring
   brand name goes into the `brand.md` mishear list (both the table and the JSON block); a
   one-off goes into `transcript/mishears.json` as `{"heard": "correct"}`. Only
   single-word, whole-word swaps are ever auto-applied.
4. **Decide the cut.** Read the transcript and write `transcript/cutsheet.json`:

   ```json
   {
     "segments": [
       {"clip": "raw/A001.mp4", "start": 12.48, "end": 17.02, "text": "the line this segment contains"},
       {"clip": "raw/A001.mp4", "start": 31.10, "end": 36.75, "text": "...", "fade_out": 0.08}
     ]
   }
   ```

   The `text` field is required. Reading the texts top to bottom must read as the finished
   video. That is the sanity check, done without watching anything. Show it to the user.
   Alongside it, propose the hook card title per the hook rule in `brand.md` (script comment
   first, else a tightened version of the strongest opening line, else no card) and show it
   to the user with the cut sheet.
   For a Short cut from long-form footage, use **montage mode** instead (next section).
5. **Motion scan, before every render** (style rule, Shaky footage):
   `scripts/motion_check.py projects/<job> [piece ...]` measures the camera's frame-to-frame motion
   in every frame of every B-roll shot of each cut sheet and writes
   `outputs/review/motion/motion-report.json`. It also fires on big subject motion, so review every
   flagged shot in frames (in sub-agents): keep subject motion, trim every camera jerk, whip,
   tilt-away or zoom so the shot starts after the camera settles and ends before it moves (0.5 s guard).
   Re-run until nothing flagged is a camera move. Never send a render that skipped this.
6. **Splice** with one filtergraph: `scripts/splice.py projects/<job>`. One trim per kept
   segment, concat, then polish the audio once on the assembled track.
7. **Remap** the transcript onto the edited timeline: `scripts/remap.py projects/<job>`.
   Writes `outputs/transcript.cut.json`. Every downstream skill reads that file. Nothing
   re-transcribes, ever.
8. **Review** with the `watch` skill in a sub-agent: every seam, plus the tail of every
   segment, not just the start.

## Montage mode (Shorts from long-form footage)

When the source is a finished long-form video, the Short is not a trim of it: the best lines
are pulled from anywhere in the transcript, re-ordered, and played over B-roll from anywhere
in the shoot. The style file's recipe says what goes where; this is the mechanics.
````

```json
{
  "mode": "montage",
  "segments": [
    {"clip": "raw/X.mp4", "start": 745.00, "end": 746.20, "text": "[SOUND: door opens]"},
    {"clip": "raw/X.mp4", "start": 7.41, "end": 10.16, "text": "This is the part nobody tells you"}
  ],
  "shots": [
    {"clip": "raw/X.mp4", "start": 745.00, "dur": 1.20, "x_offset": 0, "note": "door opens"},
    {"clip": "raw/X.mp4", "start": 36.50, "dur": 1.30, "x_offset": -120, "note": "street wide"},
    {"sync": true, "dur": 2.00, "x_offset": 60, "note": "host to camera"}
  ]
}
```

- **`segments` is the audio spine**, in play order. `remap.py` reads only these, so captions stay word-accurate. A sound-only segment has no words; give it a bracketed `text`.
- **`shots` is the picture**, in play order on the output timeline, independent of the segments: several shots can sit under one line, and one shot can straddle the join between two trimmed pieces of a line. That is how a trimmed hook's joins are hidden. The last shot's `dur` is fitted so the picture ends exactly with the audio.
- **`sync: true`** shows the speaker's own picture under the audio at that moment (lips match). Only put a sync shot over an uncut stretch of one line; the joins in a trimmed line go under B-roll.
- **B-roll is muted.** A long-form edit carries voice-over across most of its B-roll, so a shot's own sound would bring another line in with it. `"sound_db": -18` mixes the shot's sound under the voice; only use it on a window with no speech (`montage.py` warns).
- **`x_offset` per shot**, picked from a frame of that shot (see the style's reframe rules).
- **`eq` per shot** (optional): an ffmpeg `eq` grade for one shot that is over- or under-exposed, e.g. `"brightness=-0.1:contrast=1.15:saturation=1.15:gamma=0.9"`. Test it on the shot's frame first and check the share of near-white pixels drops; the style's colour rules still apply.
- **`zoom` per shot** (optional): `{"scale": 1.6, "cx": 0.46, "cy": 0.40}` crops a punch-in around that point (a share of the frame) and scales back to full size. Use it to make a reaction on a wide land (a first reaction, a laugh), or to crop a stand or tripod off a frame edge (`scale` ~1.1). Works on sync shots too. Keep `scale` at or under ~1.8 on 4K sources.
- **Check the source for burned-in text** (titles, prices, handles) before choosing a shot; a finished edit usually has it. Ask for a clean export rather than cropping around it.
- **Preview before splicing:** `scripts/montage.py projects/<job>` prints each line with its output and source times, the shots under it, the total length, and warnings. Show the user that, not the JSON.
- Cuts are frame-exact: shot boundaries are snapped to the output frame grid so thirty cuts never drift against the audio. Every audio join gets a 5 ms edge fade against clicks.

What gets cut automatically

- Filler words when they are vestigial. Stutters and false starts.
- Long pauses, per the style's `rough_cut.pauses` for this format: keep short natural pauses (for example up to ~0.25 s in a fast short), cut long ones (over ~0.3 s) down to ~0.12-0.15 s, never to zero. Without that key, cut silences over ~0.4 s (long form in the editorial style keeps up to 0.6 s so a key point can land).
- Tangents that do not serve the hook.
- Throat clears, "let me start over".
- Any preamble before the hook lands. **Every video opens on the hook.**
- When a line was recorded several times: **take the last one, always.** It is the warmest delivery and comparing takes wastes an hour.

Preserve cadence. Do not surgically remove every "like". Some of them are rhythm.

Frame rate

Every style, every format: the cut keeps the frame rate it was shot in. Never convert (25 to 30 fps duplicates every fifth frame and the motion stutters). `base.mp4` sets the rate for every later stage; in montage mode it is the main footage's rate (the first `raw/` clip), even when a B-roll sound opens the cut, and any shot from a clip at another rate (29.97 fps B-roll in a 25 fps cut) is conformed to it (`splice.py` adds an `fps` filter to that shot); graphics and B-roll pro
» be it from there. A style's `"fps": "source"`
states this; a style must not name a fixed rate.

Gotchas

- **Never pass a long filtergraph inline on macOS.** XProtect blocks ffmpeg launched with a long multi-line `-filter_complex` argument as "Malicious Script Blocked" and kills the terminal (and Claude Code) with it. Write the graph to a file and pass it with `-/filter_complex <file>` (ffmpeg 7+); `splice.py` and `mix.py` do this via `graph_args()`. Applies to any ad-hoc ffmpeg command too, including composites.

- **Stream copy does not work on arbitrary cut points.** `-c copy` desyncs audio and video. Re-encode with hardware acceleration (`splice.py` picks `h264_videotoolbox` on macOS, `libx264 -preset veryfast` elsewhere). About 15 s per minute of output.
- **Never encode audio per segment.** It rides through the cut as PCM inside the one filtergraph, then gets amplified and limited once on the assembled track. Per-segment encoding puts a click at every join.
- **Do not auto-snap cuts to silence.** Word-level alignment is the advantage. Silence

```
  detection drags deliberate boundaries into filler and awkward pauses.
- **Retake seams clip word tails.** When the speaker cuts in on their own previous word,
  the kept word sounds chopped. Extend the out point slightly into the stumble and set
  `fade_out` (seconds, ~0.05-0.12) on that segment so the word rings out.
- **Stumbles hide inside long word spans.** WhisperX sometimes merges a stumble and its
  retake into one word span over 1.2 s. If a word's duration looks wrong for what it should
  sound like, run silence detection across just that span before deciding the cut:
  `ffmpeg -ss <s> -to <e> -i <clip> -af silencedetect=n=-35dB:d=0.15 -f null -`.
  `unknown_words.py --long-spans` lists them.
- **The transcript will mishear things.** Cross-check before killing a line. "Claude"
  becoming "cloud" makes a good sentence look broken.
- **Screen recordings can carry a chapter track** that inflates reported duration and leaves
  a black tail. Probe with `ffprobe`; `splice.py` strips chapters (`-map_chapters -1`).

## Scripts

- `scripts/transcribe.sh <job>`: WhisperX in an isolated uv environment (never system
  Python). Skips anything already transcribed.
- `scripts/unknown_words.py <job> [--long-spans]`
- `scripts/splice.py <job> [--dry-run] [--encoder X] [--style S --format F]`: with a style
  whose `rough_cut.reframe` covers the format, reframes widescreen onto the vertical canvas;
  each cut sheet segment may then carry `x_offset` (px, positive = picture moves right).
- `scripts/montage.py <job>`: preview and validate a montage cut sheet (`splice.py` runs the
  same validation).
- `scripts/remap.py <job>`
- `scripts/motion_check.py <job> [piece ...]`: camera motion in every frame of every B-roll shot;
  flags jerks/whips (review flagged shots in frames: subject motion is fine)
- `scripts/beat_snap.py <job> --first-beat T --beat S [--quarter i,j] [--fixed i] [--at i=T] [--write]`:
  move the picture cuts onto the music's grid
- `scripts/words.py`, `scripts/sound_scan.py`, `scripts/strip.py`: word gaps, band levels
  (voice vs other sound), labelled frame strips
```

FILE: .claude/skills/rough-cut/scripts/beat_snap.py

3,866 bytes

```
#!/usr/bin/env python3
"""Move a montage cut's picture cuts onto the music's beat grid (the style's "cut on the beat").

The grid on the video timeline is first_beat + n * beat (from the music plan: the video time
where the track's beat lands, and the beat length). Each cut goes to the nearest eighth, or the
nearest quarter for the shots listed in --quarter (section changes, key cuts), if that is
within --max seconds; otherwise it stays and is flagged. Cuts before the music starts stay put.
Only shot durations change: a B-roll shot keeps its source in-point, a sync shot keeps
following its line. It warns when a sync shot would run outside its line.
Usage: beat_snap.py projects/<job> --first-beat 1.80 --beat 0.43478 [--quarter 2,5] [--max 0.3]
       [--fixed 7] [--at 21=25.28] [--write]    (shot indices as montage.py prints them, from 0)
"""
import argparse
import json
from pathlib import Path

import montage


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("job", type=Path)
    ap.add_argument("--first-beat", type=float, required=True)
    ap.add_argument("--beat", type=float, required=True)
    ap.add_argument("--quarter", default="", help="shot indices whose END cut goes on a quarter")
    ap.add_argument("--fixed", default="", help="shot indices whose END cut must not move")
    ap.add_argument("--max", type=float, default=0.3)
    ap.add_argument("--at", action="append", default=[], help="n=T: put shot n's END cut at T (e.g. on an ending hi
» t)")
    ap.add_argument("--write", action="store_true")
    a = ap.parse_args()

    sheet = montage.load_sheet(a.job)
    fps = montage.output_fps(a.job, sheet)
    pic, total = montage.picture_timeline(sheet, fps)
    quarter = {int(x) for x in a.quarter.split(",") if x}
    fixed = {int(x) for x in a.fixed.split(",") if x}
    at = {int(k): float(v) for k, v in (x.split("=") for x in a.at)}
    atl = montage.audio_timeline(sheet["segments"])

    cuts = []
    for n, (o0, o1, clip, src, nf, sh) in enumerate(pic[:-1]):
        res = a.beat if n in quarter else a.beat / 2
        new, flag = o1, ""
        if n in at:
            new, flag = float(round(at[n] * fps) / fps), "placed by hand"
        elif n in fixed:
            flag = "fixed"
```

```
        elif o1 < a.first_beat - 0.1:
            flag = "before the music"
        else:
            target = a.first_beat + round((o1 - a.first_beat) / res) * res
            target = round(target * fps) / fps
            if abs(target - o1) <= a.max + 1e-6:
                new = float(target)
            else:
                flag = f"OFF GRID (nearest {target:.2f} is {target - o1:+.2f}s away)"
        cuts.append((new, flag))

    prev, durs = 0.0, []
    print(f"{'#':>2} {'cut':>6} -> {'new':>6}  {'shift':>6}  {'grid':>7}  shot")
    for n, (o0, o1, clip, src, nf, sh) in enumerate(pic):
        end = cuts[n][0] if n < len(cuts) else float(round(total * fps) / fps)
        durs.append(round(end - prev, 4))
        if n < len(cuts):
            kind = "quarter" if n in quarter else "eighth"
            print(f"{n:2d} {o1:6.2f} -> {end:6.2f}  {end - o1:+6.2f}  {kind:>7}  "
                  f"{sh.get('note', '')[:50]} {cuts[n][1]}")
        if sh.get("sync"):
            a0, a1, s = max(atl, key=lambda x: min(x[1], end) - max(x[0], prev))
            if prev < a0 - 0.5 / float(fps) or end > a1 + 0.5 / float(fps):
                print(f"   WARNING: sync shot {n} ({prev:.2f}-{end:.2f}) runs outside its line "
                      f"({a0:.2f}-{a1:.2f})")
        prev = end
    if a.write:
        for sh, d in zip(sheet["shots"], durs):
            sh["dur"] = d
        sheet["shots"][-1].pop("dur", None)  # the last shot is fitted to the audio
        (a.job / "transcript" / "cutsheet.json").write_text(json.dumps(sheet, indent=1))
        print("written")


if __name__ == "__main__":
    main()
```

FILE: .claude/skills/rough-cut/scripts/montage.py

9,552 bytes

```
#!/usr/bin/env python3
"""Montage mode: lines from anywhere in a long-form transcript, played over re-ordered B-roll.

A montage cut sheet has two tracks:

  segments  the AUDIO spine, as in a plain cut sheet: each line (or trimmed piece of one)
            in play order. remap.py reads only these, so captions stay word-accurate.
  shots     the PICTURE, in play order on the output timeline, independent of the segments,
            so one line can carry several shots and a shot can hide the join between two
            trimmed pieces of a line. Each shot:
              {"clip", "start", "dur", "x_offset", "note"}      B-roll, muted
              {"sync": true, "dur", "x_offset", "note"}          the speaker's own picture,
                                                                 lip-synced to the audio
              optional "sound_db": the shot's own sound mixed under the voice at that level.
            The last shot's dur is fitted so the picture ends exactly with the audio.

Run directly to print the plan (lines, the shots under each, total length) and validate it:
  montage.py projects/<job> [--fps 25]
"""
import json
import subprocess
import sys
from fractions import Fraction
from pathlib import Path

EDGE_FADE = 0.005  # s; de-clicks every audio join without touching the words


def load_sheet(job: Path) -> dict:
    return json.loads((job / "transcript" / "cutsheet.json").read_text())


def is_montage(sheet: dict) -> bool:
    return sheet.get("mode") == "montage"


def probe_fps(path: Path) -> Fraction:
    out = subprocess.run(["ffprobe", "-v", "error", "-select_streams", "v:0", "-show_entries",
                          "stream=r_frame_rate", "-of", "csv=p=0", str(path)],
                         capture_output=True, text=True, check=True).stdout.strip()
    # Some DJI camera files report v:0 more than once (stream groups); the first line is the camera.
    if not out:   # audio-only (a voiceover recording used as a segment): no picture rate; never used for picture
        return Fraction(25)
```

```
    return Fraction(out.splitlines()[0].split(",")[0])


def output_fps(job: Path, sheet: dict) -> Fraction:
    """The cut's frame rate: the main footage's (the first clip in raw/), never a B-roll clip's,
    even when a B-roll sound opens the cut. A sheet may pin it with "fps"."""
    if sheet.get("fps"):
        return Fraction(str(sheet["fps"]))
    clips = [s["clip"] for s in sheet["segments"]]
    main = next((c for c in clips if c.startswith("raw/")), clips[0])
    return probe_fps(job / main)


def audio_timeline(segs):
    """[(out_start, out_end, seg)] for the audio spine."""
    t, out = 0.0, []
    for s in segs:
        d = float(s["end"]) - float(s["start"])
        out.append((t, t + d, s))
        t += d
    return out


def picture_timeline(sheet: dict, fps: Fraction):
    """Resolve shots to frame-exact windows: [(out_start, out_end, clip, src_start, n_frames, shot)].

    Cuts land on the output frame grid, so ~30 shots never drift against the audio.
    Source starts are snapped to the source frame grid (same rate as the output here).
    """
    segs = sheet["segments"]
    atl = audio_timeline(segs)
    total = atl[-1][1] if atl else 0.0
    total_frames = round(total * fps)
    shots = sheet.get("shots", [])
    if not shots:
        sys.exit("montage cut sheet has no shots")

    out, t, frame = [], 0.0, 0
    for n, sh in enumerate(shots):
        last = n == len(shots) - 1
        end_frame = total_frames if last else round((t + float(sh["dur"])) * fps)
        nf = end_frame - frame
        if nf <= 0:
            sys.exit(f"shot {n} has no frames left (picture already covers the audio): {sh}")
        o_start, o_end = float(frame / fps), float(end_frame / fps)
        if sh.get("sync"):
            # The line it overlaps most: frame snapping can start it a hair before its line.
            a, b, s = max(atl, key=lambda x: min(x[1], o_end) - max(x[0], o_start))
            if min(b, o_end) <= max(a, o_start):
                sys.exit(f"sync shot {n} lies outside the audio: {sh}")
            clip, src = s["clip"], float(s["start"]) + (o_start - a)
            if o_end > b + 0.5 / float(fps):
                print(f"WARNING: sync shot {n} runs past the end of its line by "
                      f"{o_end - b:.2f}s; lips drift after the join", file=sys.stderr)
        else:
            clip, src = sh["clip"], float(sh["start"])
        src = round(src * fps) / fps  # snap to the source frame grid
        out.append((o_start, o_end, clip, float(src), nf, sh))
        t, frame = o_end, end_frame
    return out, total


def words_in(transcript: dict, clip: str, a: float, b: float):
    for seg in transcript.get(clip, {}).get("segments", []):
        for w in seg.get("words", []):
            if "start" in w and w["end"] > a and w["start"] < b:
                yield w


def validate(job: Path, sheet: dict, fps: Fraction):
    """Hard errors exit; soft problems come back as warnings."""
    warn = []
    for s in sheet["segments"]:
        for k in ("clip", "start", "end", "text"):
            if k not in s:
                sys.exit(f"segment missing '{k}': {s}")
        if float(s["end"]) <= float(s["start"]):
            sys.exit(f"segment end <= start: {s}")
    for n, sh in enumerate(sheet.get("shots", [])):
        if not sh.get("sync"):
            for k in ("clip", "start"):
                if k not in sh:
                    sys.exit(f"shot {n} missing '{k}': {sh}")
        if "dur" not in sh and n != len(sheet["shots"]) - 1:
```

```
            sys.exit(f"shot {n} missing 'dur': {sh}")
    pic, total = picture_timeline(sheet, fps)

    # A shot that asks for more frames than its clip has renders short, silently: every later
    # shot then lands early and sync shots drift off their audio. Refuse it.
    lengths = {}
    for n, (a, b, clip, src, nf, sh) in enumerate(pic):
        if clip not in lengths:
            out = subprocess.run(["ffprobe", "-v", "error", "-show_entries", "format=duration",
                                  "-of", "csv=p=0", str(job / clip)],
                                 capture_output=True, text=True, check=True).stdout
            lengths[clip] = float(out.splitlines()[0])
        r = probe_fps(job / clip)
        slow = r > fps and not sh.get("sync") and not sh.get("realtime")
        used = nf / float(r if slow else fps)  # faster B-roll plays every frame unless "realtime"
        if src + used > lengths[clip] + 0.5 / float(fps):
            sys.exit(f"shot {n} needs {clip} up to {src + used:.2f}s but the clip is "
                     f"{lengths[clip]:.2f}s long: {sh}")
    # Same for the audio spine: a segment past its clip's end comes up short and every later
    # line lands early against the picture.
    for s in sheet["segments"]:
        clip = s["clip"]
        if clip not in lengths:
            out = subprocess.run(["ffprobe", "-v", "error", "-show_entries", "format=duration",
                                  "-of", "csv=p=0", str(job / clip)],
                                 capture_output=True, text=True, check=True).stdout
            lengths[clip] = float(out.splitlines()[0])
        if float(s["end"]) > lengths[clip] + 0.02:
            sys.exit(f"segment needs {clip} up to {float(s['end']):.2f}s but the clip is "
                     f"{lengths[clip]:.2f}s long: {s.get('text', '')[:60]}")

    tpath = job / "transcript" / "transcript.json"
    transcript = json.loads(tpath.read_text())["clips"] if tpath.exists() else {}
    for n, (a, b, clip, src, nf, sh) in enumerate(pic):
        dur = nf / float(fps)
        if dur < 0.4:
            warn.append(f"shot {n} is {dur:.2f}s: shorter than a readable cut")
        if "sound_db" in sh:
            spoken = list(words_in(transcript, clip, src, src + dur))
            if spoken and not sh.get("sync"):
                said = " ".join(w["word"] for w in spoken)
                warn.append(f"shot {n} carries sound_db but its window has speech: \"{said}\"")
    return pic, total, warn


def fmt(t: float) -> str:
    m, s = divmod(t, 60)
    return f"{int(m)}:{s:05.2f}"


def print_plan(sheet: dict, pic, total: float, warn):
    atl = audio_timeline(sheet["segments"])
    print(f"{'OUT':>13}  {'SOURCE':>17}  LINE / SHOTS UNDER IT")
    for a, b, s in atl:
        src = f"{fmt(float(s['start']))}-{fmt(float(s['end']))}"
        print(f"{a:5.2f}-{b:5.2f}s  {src:>17}  \"{s['text']}\"")
        for pa, pb, clip, psrc, nf, sh in pic:
            lo, hi = max(a, pa), min(b, pb)
            if hi - lo > 1e-3:
                kind = "SYNC" if sh.get("sync") else "shot"
                snd = f"  sound {sh['sound_db']} dB" if "sound_db" in sh else ""
                xo = sh.get("x_offset", 0) or 0
                print(f"{'':32}  {kind} {pa:5.2f}-{pb:5.2f}  src {fmt(psrc)}  "
                      f"x{xo:+d}  {sh.get('note', '')}{snd}")
    print(f"\nTotal: {total:.2f}s  ({len(atl)} audio segments, {len(pic)} shots)")
    for w in warn:
        print("WARNING:", w)


def main():
    if len(sys.argv) < 2:
        sys.exit("usage: montage.py projects/<job> [--fps N]")
    job = Path(sys.argv[1])
    sheet = load_sheet(job)
    if not is_montage(sheet):
        sys.exit('cut sheet is not "mode": "montage"')
    if "--fps" in sys.argv:
        fps = Fraction(sys.argv[sys.argv.index("--fps") + 1])
    else:
        fps = output_fps(job, sheet)
    pic, total, warn = validate(job, sheet, fps)
    print_plan(sheet, pic, total, warn)
```

```
if __name__ == "__main__":
    main()
```

FILE: .claude/skills/rough-cut/scripts/motion_check.py

3,360 bytes

```
"""Style rule (no shaky footage in the cut): motion measured in every shot, not sampled.
For every B-roll shot in every cut sheet, read every frame of the span the cut uses (plus 0.5 s either
side for context), measure the global camera translation frame to frame (phase correlation on a
480 px greyscale copy) and flag jerks/whips: a frame-to-frame jump far above the shot's own median, or
an absolute jump over JUMP px. Reports where in the shot (first/last 0.5 s matter most)."""
import json, sys, cv2, numpy as np
from pathlib import Path
JOB = Path(sys.argv[1]).resolve() if len(sys.argv) > 1 and Path(sys.argv[1]).is_dir() else Path.cwd()
# usage: python3 motion_check.py <job dir> [piece ...]   (run from anywhere)
JUMP = 9.0          # px per frame at 480 px wide: a whip/jerk, not a pan
REL = 4.0           # x the shot's median motion
args = [a for a in sys.argv[1:] if not Path(a).is_dir()]
pieces = args or sorted(p.stem.replace("cutsheet-", "") for p in (JOB / "transcript").glob("cutsheet-*.json"))
def fps_of(p):
    c = cv2.VideoCapture(str(p)); f = c.get(cv2.CAP_PROP_FPS); c.release(); return f or 25.0
report = []
for piece in pieces:
    sheet = json.load(open(JOB / f"transcript/cutsheet-{piece}.json"))
    t = 0.0
    for sh in sheet["shots"]:
        dur = sh["dur"]; out0 = t; t += dur
        if sh.get("sync") or not sh.get("clip"): continue
        clip = JOB / sh["clip"]; r = fps_of(clip)
        span = dur if (sh.get("realtime") or r <= 25.5) else dur * 25.0 / r
        s0 = sh["start"]
        cap = cv2.VideoCapture(str(clip)); cap.set(cv2.CAP_PROP_POS_MSEC, max(0, s0) * 1000)
        prev = None; mags = []; times = []
        n = int(round(span * r))
        for i in range(n + 1):
            ok, fr = cap.read()
            if not ok: break
            g = cv2.cvtColor(cv2.resize(fr, (480, int(480 * fr.shape[0] / fr.shape[1]))), cv2.COLOR_BGR2GRAY).astyp
» e(np.float32)
            if prev is not None:
                (dx, dy), _ = cv2.phaseCorrelate(prev, g)
                mags.append(float(np.hypot(dx, dy)) * r / 25.0)   # per output (25 fps) frame
                times.append(i / r)
            prev = g
        cap.release()
        if not mags: continue
        m = np.array(mags); med = float(np.median(m)) + 0.3
        bad = [(times[k], m[k]) for k in range(len(m)) if m[k] > JUMP or m[k] > REL * med and m[k] > 4.0]
        if bad:
            edge = any(tt < 0.5 or tt > span - 0.5 for tt, _ in bad)
            worst = max(bad, key=lambda x: x[1])
            report.append({"piece": piece, "out": round(out0, 2), "clip": sh["clip"].split("/")[-1], "start": s0,
                           "span": round(span, 2), "median": round(med, 1), "worst_at": round(worst[0], 2),
                           "worst_px": round(worst[1], 1), "n_bad": len(bad), "at_edge": edge,
                           "bad_times": [round(b[0], 2) for b in bad][:12], "note": sh["note"][:70]})
(JOB / "outputs/review/motion").mkdir(parents=True, exist_ok=True)
out = JOB / "outputs/review/motion/motion-report.json"
json.dump(report, open(out, "w"), indent=1)
for r_ in report:
    print(f"{r_['piece']:6} @{r_['out']:7.2f} {r_['clip']:28} src {r_['start']:7} worst {r_['worst_px']:5.1f}px at
» +{r_['worst_at']:5.2f}/{r_['span']} {'EDGE' if r_['at_edge'] else ''} n={r_['n_bad']}  {r_['note']}")
print(len(report), "shots flagged")
```

FILE: .claude/skills/rough-cut/scripts/remap.py

3,281 bytes

```
#!/usr/bin/env python3
"""Remap the source transcript onto the edited timeline.

Reads transcript/transcript.json + transcript/cutsheet.json, applies the mishear
lists (brand.md, then transcript/mishears.json), and writes
outputs/transcript.cut.json: {"duration": s, "words": [{"word","start","end","segment"}]}.
Every downstream skill reads that file. Nothing re-transcribes.

Only single-word, whole-word swaps are applied: changing the word count would break
every timestamp downstream.
"""
import json
import re
import sys
from pathlib import Path
```

```
PUNCT = re.compile(r"^(\W*)(.*?)(\W*)$")


def find_brand(start: Path) -> Path:
    for d in [start.resolve(), *start.resolve().parents]:
        if (d / "brand.md").exists():
            return d / "brand.md"
    sys.exit("brand.md not found above the job folder")


def brand_json(brand: Path) -> dict:
    m = re.search(r"```json\s*(\{.*?\})\s*```", brand.read_text(), re.S)
    return json.loads(m.group(1)) if m else {}


def load_mishears(job: Path) -> dict:
    fixes = {k.lower(): v for k, v in brand_json(find_brand(job)).get("mishears", {}).items()}
    local = job / "transcript" / "mishears.json"
    if local.exists():
        fixes.update({k.lower(): v for k, v in json.loads(local.read_text()).items()})
    # Enforce single-word -> single-word only.
    return {k: v for k, v in fixes.items() if len(k.split()) == 1 and len(v.split()) == 1}


def fix_word(word: str, fixes: dict) -> str:
    lead, core, trail = PUNCT.match(word.strip()).groups()
    rep = fixes.get(core.lower())
    return f"{lead}{rep}{trail}" if rep else word.strip()


def clip_words(clip_json: dict):
    for seg in clip_json.get("segments", []):
        for w in seg.get("words", []):
            if "start" in w and "end" in w:  # WhisperX leaves some tokens unaligned
                yield w


def main():
    job = Path(sys.argv[1] if len(sys.argv) > 1 else sys.exit("usage: remap.py projects/<job>"))
    transcript = json.loads((job / "transcript" / "transcript.json").read_text())["clips"]
    segs = json.loads((job / "transcript" / "cutsheet.json").read_text())["segments"]
    fixes = load_mishears(job)

    out, offset = [], 0.0
    for n, s in enumerate(segs):
        start, end = float(s["start"]), float(s["end"])
        if s.get("text", "").startswith("["):  # a sound ("[SOUND: ...]"), not speech: no words
            offset += end - start
            continue
        # A B-roll sound (a natural-sound open from broll/) may have no transcript: it has no words.
        for w in clip_words(transcript.get(s["clip"], {})):
            ws, we = float(w["start"]), float(w["end"])
            if we <= start or ws >= end:
                continue
            out.append({
                "word": fix_word(w["word"], fixes),
                "start": round(max(ws, start) - start + offset, 3),
                "end": round(min(we, end) - start + offset, 3),
                "segment": n,
            })
        offset += end - start

    dest = job / "outputs" / "transcript.cut.json"
    dest.parent.mkdir(parents=True, exist_ok=True)
    dest.write_text(json.dumps({"duration": round(offset, 3), "words": out}, indent=1))
    print(f"Wrote {dest}: {len(out)} words, {offset:.2f}s")


if __name__ == "__main__":
    main()
```

FILE: .claude/skills/rough-cut/scripts/sound_scan.py

1,965 bytes

```
#!/usr/bin/env python3
"""Scan a stretch of audio in 20 ms steps: overall level, low band (80-400 Hz, voiced hum) and
high band (>2.5 kHz, slurp/crunch/sizzle noise).

Answers the questions a fragment or montage cut needs: where a word really starts (WhisperX can
stretch a word over the silence before it), whether a moment is live sound (room tone
under it) or voice-over / silent B-roll (digital silence, -180 dB), and whether a closing sound
is a sip or crunch (high band) or a voiced "mm" (low band only).
```

```
Usage: sound_scan.py <media> <start_s> <dur_s> [--every N]   (print every Nth row)
"""
import array
import math
import subprocess
import sys


def band(media, a0, dur, af):
    raw = subprocess.run(["ffmpeg", "-v", "error", "-ss", str(a0), "-t", str(dur), "-i", media, "-ac", "1",
                          "-ar", "16000", *(["-af", af] if af else []), "-f", "s16le", "-"],
                         capture_output=True, check=True).stdout
    x = array.array("h")
    x.frombytes(raw)
    hop = 320
    return [20 * math.log10(math.sqrt(sum(v * v for v in x[i:i + hop]) / hop) / 32768 + 1e-9)
            for i in range(0, len(x) - hop, hop)]


def main():
    if len(sys.argv) < 4:
        sys.exit("usage: sound_scan.py <media> <start_s> <dur_s> [--every N]")
    media, a0, dur = sys.argv[1], float(sys.argv[2]), float(sys.argv[3])
    every = int(sys.argv[sys.argv.index("--every") + 1]) if "--every" in sys.argv else 1
    full = band(media, a0, dur, None)
    low = band(media, a0, dur, "highpass=f=80,lowpass=f=400")
    high = band(media, a0, dur, "highpass=f=2500")
    for i, (f, l, h) in enumerate(zip(full, low, high)):
        if i % every:
            continue
        tag = ("silent" if f < -100 else
               "HUM" if l > -45 and l - h > 8 else
               "noise" if h > -50 and h >= l - 3 else "")
        print(f"{a0 + i * 0.02:8.2f}  all {f:6.1f}  low {l:6.1f}  high {h:6.1f}  {tag}")


if __name__ == "__main__":
    main()
```

FILE: .claude/skills/rough-cut/scripts/splice.py

21,772 bytes

```
#!/usr/bin/env python3
"""Splice transcript/cutsheet.json into outputs/base.mp4 with ONE FFmpeg filtergraph.

One trim per kept segment, concat, then amplify + limit the audio once on the
assembled track. Audio stays PCM inside the graph, so it is encoded exactly once.
Video is re-encoded (stream copy desyncs on arbitrary cut points).

With --style/--format, a style whose rough_cut.reframe has an entry for the format puts
widescreen source on a vertical canvas: scale to picture_height, crop to the canvas width
(shifted per segment by x_offset px, positive = picture moves right), pad with black bars.

A cut sheet with "mode": "montage" takes its audio from the segments and its picture from
a separate shots track (see montage.py).
"""
import argparse
import json
import re
import platform
import subprocess
import sys
import tempfile
from pathlib import Path

import montage


def graph_args(graph: str, path: Path):
    """ffmpeg args that read the filtergraph from a file, never inline on the command line.

    macOS XProtect blocks ffmpeg launched with a long multi-line filtergraph argument as
    "Malicious Script Blocked" (and kills the terminal with it). A graph file avoids that.
    """
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(graph)
    ver = subprocess.run(["ffmpeg", "-hide_banner", "-version"],
                         capture_output=True, text=True).stdout
    m = re.search(r"ffmpeg version n?(\d+)\.", ver)
    if m and int(m.group(1)) < 7:
        return ["-filter_complex_script", str(path)]
    return ["-/filter_complex", str(path)]
```

```
def default_encoder():
    if platform.system() == "Darwin":
        return ["-c:v", "h264_videotoolbox", "-b:v", "40M"]  # 4K masters: 16M blocked up visibly
    return ["-c:v", "libx264", "-preset", "veryfast", "-crf", "18"]


def root_of(job: Path) -> Path:
    for d in [job.resolve(), *job.resolve().parents]:
        if (d / "brand.md").exists():
            return d
    sys.exit("brand.md not found above the job folder")


def probe_size(path: Path):
    out = subprocess.run(["ffprobe", "-v", "error", "-select_streams", "v:0", "-show_entries",
                          "stream=width,height", "-of", "csv=p=0", str(path)],
                         capture_output=True, text=True, check=True).stdout.strip()
    w, h = out.splitlines()[0].split(",")[:2]
    return int(w), int(h)


def reframe_filter(size, cfg, x_offset):
    """scale -> crop (shifted, clamped to fill the width) -> pad to the canvas."""
    cw, ch = cfg["canvas"]
    ph = int(cfg["picture_height"])
    iw, ih = size
    sw = max(cw, round(iw * ph / ih / 2) * 2)
    x = min(max((sw - cw) // 2 - int(round(x_offset)), 0), sw - cw)
    return (f"scale={sw}:{ph},crop={cw}:{ph}:{x}:0,"
            f"pad={cw}:{ch}:0:{(ch - ph) // 2}:black,setsar=1")


def zoom_filter(size, z):
    """Crop a 1/scale window around (cx, cy), clamped inside the frame, and scale back to size.

    For a punch-in on a wide (a reaction, a key moment). Keep scale at or under ~1.8 on 4K sources:
    beyond that the upscale softens visibly.
    """
    iw, ih = size
    s = float(z["scale"])
    cw, ch = round(iw / s / 2) * 2, round(ih / s / 2) * 2
    x = min(max(round(float(z.get("cx", 0.5)) * iw - cw / 2), 0), iw - cw)
    y = min(max(round(float(z.get("cy", 0.5)) * ih - ch / 2), 0), ih - ch)
    return f"crop={cw}:{ch}:{x}:{y},scale={iw}:{ih}:flags=lanczos,setsar=1"


def build_montage(job: Path, sheet: dict, encoder_args, reframe=None):
    """Audio from the segments (the spine), picture from the shots (see montage.py)."""
    segs = sheet["segments"]
    fps = montage.output_fps(job, sheet)
    pic, total, warn = montage.validate(job, sheet, fps)
    for w in warn:
        print("WARNING:", w, file=sys.stderr)

    # Audio: one shared input per clip (cheap to buffer). Picture: one input PER SHOT, seeked
    # to just before its in-point. Trimming every shot out of one shared input makes ffmpeg hold
    # decoded 4K frames for every later branch while it decodes up to an earlier-needed one; an
    # edit that jumps around a long take (a cold open from its end, then back to its start) then
    # runs out of memory and is killed.
    clips = []
    for c in [s["clip"] for s in segs] + [p[2] for p in pic]:
        if c not in clips:
            clips.append(c)
    for c in clips:
        if not (job / c).exists():
            sys.exit(f"missing clip: {job / c}")
    # A segment may take its sound from another file while its picture (sync shots) stays on "clip":
    # "audio": path, "audio_start": the clip time that file's 0 s lines up with (e.g. a stem separated from
    # clip 4.0 s onwards has audio_start 4.0). Used for music-free stems (copyright) and similar cleanups.
    asrc = lambda s: s.get("audio") or s["clip"]
    for s in segs:
        if s.get("audio") and not (job / s["audio"]).exists():
            sys.exit(f"missing audio: {job / s['audio']}")
    inputs, aidx = [], {}
    for c in [asrc(s) for s in segs]:
        if c not in aidx:
            aidx[c] = len(aidx)
            inputs += ["-i", str(job / c)]
    sizes = {c: probe_size(job / c) for c in clips} if reframe else {}
    rates = {c: montage.probe_fps(job / c) for c in clips}

    parts, alabels, vlabels, beds = [], [], [], []
    e = montage.EDGE_FADE
```

```
    # "audio_xfade" (s, sheet level): every audio join is a crossfade of that length centred on the cut, so
    # room tone and street noise blend from one piece to the next instead of jumping (no harsh audio
    # switches). Each piece borrows half the crossfade from its source on both sides, so the total length
    # and every cut point stay exactly where the cut sheet puts them.
    xf = float(sheet.get("audio_xfade", 0) or 0) if len(segs) > 1 else 0.0
    for n, s in enumerate(segs):
        i = aidx[asrc(s)]
        shift = float(s.get("audio_start", 0) or 0) if s.get("audio") else 0.0
        start, end = float(s["start"]) - shift, float(s["end"]) - shift
        pre = xf / 2 if (xf and n > 0) else 0.0
        post = xf / 2 if (xf and n < len(segs) - 1) else 0.0
        # Every piece must come out exactly (end - start) + pre + post long, or every later join drifts out
        # of sync (seen in a real render: a handle cut short at two clip ends put the sound 160 ms ahead of the
        # lips for the rest of the video). A handle that runs past either end of the clip's audio is filled
        # with silence instead of being borrowed from elsewhere in the clip.
        dur = end - start + pre + post
        head = max(pre - start, 0.0)
        start, end = max(start - pre, 0.0), end + post
        if head and float(s.get("gain_db", 0) or 0) > -60:
            print(f"note: segment {n} starts {s['start'] - shift:.3f}s into its audio; {head:.3f}s of its "
                  f"crossfade handle is silence", file=sys.stderr)
        fade = max(float(s.get("fade_out", 0) or 0), e if not post else 0)
        fin = max(float(s.get("fade_in", 0) or 0), e if not pre else 0)  # e.g. ease a J-cut's street noise in
        gain = f"volume={float(s['gain_db'])}dB," if s.get("gain_db") else ""  # e.g. a faint natural sound
        if s.get("afilter"):  # a per-segment audio filter, e.g. notches for a beep under one take
            gain += s["afilter"].rstrip(",") + ","
        fades = (f"afade=t=in:d={fin:.3f}," if fin else "") + (f"afade=t=out:st={max(dur - fade, 0):.3f}:d={fade:.3
» f}," if fade else "")
        delay = f"adelay=delays={head * 1000:.0f}:all=1," if head else ""
        parts.append(f"[{i}:a]atrim=start={start:.3f}:end={end:.3f},asetpts=PTS-STARTPTS,{delay}"
                     f"apad=whole_dur={dur:.3f},atrim=end={dur:.3f},{gain}{fades}anull[a{n}]")
        alabels.append(f"[a{n}]")
    if xf:
        prev = alabels[0]
        for n, lab in enumerate(alabels[1:], 1):
            out = "[voice]" if n == len(alabels) - 1 else f"[ax{n}]"
            parts.append(f"{prev}{lab}acrossfade=d={xf:.3f}:c1=tri:c2=tri{out}")
            prev = out
    else:
        parts.append("".join(alabels) + f"concat=n={len(segs)}:v=0:a=1[voice]")

    half = 0.5 / float(fps)
    dissolves = any(p[5].get("dissolve") for p in pic)
    orig_nf = [p[4] for p in pic]
    for n, (o_start, o_end, clip, src, nf, sh) in enumerate(pic):
        r = rates[clip]
        if n and sh.get("dissolve"):
            # Cross-dissolve in from the previous shot: take d extra seconds from before this
            # shot's in-point and overlap them on the previous shot, so the cut's timing (and
            # every later shot's sync) stays exactly where it was.
            dzf = round(float(sh["dissolve"]) * float(fps))
            src -= dzf / float(fps)
            nf += dzf
        slow = r > fps and not sh.get("sync") and not sh.get("realtime")
        used = nf / float(r if slow else fps)
        seek = max(src - 0.5, 0.0)
        i = len(aidx) + n
        inputs += ["-ss", f"{seek:.4f}", "-t", f"{src - seek + used + 0.5:.4f}", "-i", str(job / clip)]
        src -= seek  # times below are now relative to this shot's own input
        if slow:
            # Picture-only B-roll shot faster than the cut (29.97 into 25): play every source
            # frame at the cut's rate, a gentle slow-motion, instead of dropping frames (judder).
            # "realtime": true on the shot plays it at normal speed instead (conformed below).
            sh_half = 0.5 / float(r)
            v = (f"[{i}:v]trim=start={src - sh_half:.4f}:end={src + nf / float(r) - sh_half:.4f},"
                 f"setpts=N/({fps.numerator}/{fps.denominator}*TB)")
        else:
            v = (f"[{i}:v]trim=start={src - half:.4f}:end={src + nf / float(fps) - half:.4f},"
                 f"setpts=PTS-STARTPTS")
            if r != fps:  # synced to its own sound: conform by dropping/repeating frames
                v += f",fps={fps.numerator}/{fps.denominator}"
        # Exactly nf frames per shot: a conform can come up a frame short, which would slide every
        # later shot early against the audio (lips out of sync). Clone the tail, cut to the count.
        v += f",tpad=stop_mode=clone:stop=2,trim=end_frame={nf},setpts=PTS-STARTPTS"
        if sh.get("eq"):  # per-shot grade, e.g. "brightness=-0.1:contrast=1.15" for a blown-out shot
            v += f",eq={sh['eq']}"
        if sh.get("vf"):  # any per-shot filter, e.g. highlight recovery or "deshake,crop=..." on a shaky insert
            v += "," + sh["vf"]
        if sh.get("zoom"):  # punch-in: {"scale": 1.6, "cx": 0.46, "cy": 0.40}, centre as a share of the frame
            v += "," + zoom_filter(probe_size(job / clip), sh["zoom"])
        grade = sheet.get("grade") or {}
        for prefix, match in (grade.get("match") or {}).items():  # per-camera match, e.g. {"raw/CAM-B/": "eq=..."}
            if clip.startswith(prefix):
                v += "," + match
```

```
        # The video's look, after the camera match; "look_skip" lists clip prefixes it is not applied to.
        if grade.get("look") and not any(clip.startswith(p) for p in grade.get("look_skip") or []):
            v += "," + grade["look"]
        if sh.get("extra_vf"):  # a per-shot adjustment after the shared look (e.g. boost one colour range)
            v += "," + sh["extra_vf"]
        if reframe:
            v += "," + reframe_filter(sizes[clip], reframe, sh.get("x_offset", 0) or 0)
        if dissolves:  # xfade needs every stream on the same rate, timebase and pixel format
            v += f",fps={fps.numerator}/{fps.denominator},settb=AVTB,format=yuv420p,setsar=1"
        parts.append(v + f"[v{n}]")
        vlabels.append(f"[v{n}]")
        if "sound_db" in sh and not sh.get("sync"):
            d = nf / float(fps)
            parts.append(f"[{i}:a]atrim=start={src:.3f}:end={src + d:.3f},asetpts=PTS-STARTPTS,"
                         f"volume={float(sh['sound_db'])}dB,afade=t=in:d={e},"
                         f"afade=t=out:st={max(d - e, 0):.3f}:d={e},"
                         f"adelay={round(o_start * 1000)}:all=1[n{n}]")
            beds.append(f"[n{n}]")
    # Picture and sound end on the same frame: the concat can come up a frame short, so clone the
    # last frame and cut to the exact count; the sound is cut there too, with a short fade, so the
    # loop restarts clean.
    n_frames = sum(p[4] for p in pic)
    end = n_frames / float(fps)
    if not dissolves:
        parts.append("".join(vlabels) + f"concat=n={len(pic)}:v=1:a=0,"
                     f"tpad=stop_mode=clone:stop=2,trim=end_frame={n_frames}[vc]")
    else:
        # Hard cuts are concatenated in runs; a dissolve shot is xfaded onto the run before it.
        run, cur, acc = [vlabels[0]], None, 0
        for n in range(1, len(pic) + 1):
            if n < len(pic) and not pic[n][5].get("dissolve"):
                run.append(vlabels[n])
                continue
            lab = f"[r{n}]"
            src_labels = ([cur] if cur else []) + run
            parts.append("".join(src_labels) + f"concat=n={len(src_labels)}:v=1:a=0{lab}")
            acc = sum(orig_nf[:n])
            cur = lab
            if n == len(pic):
                break
            d = round(float(pic[n][5]["dissolve"]) * float(fps)) / float(fps)
            xl = f"[x{n}]"
            parts.append(f"{cur}{vlabels[n]}xfade=transition=fade:duration={d:.4f}:"
                         f"offset={acc / float(fps) - d:.4f}{xl}")
            cur, run = xl, []
        parts.append(f"{cur}tpad=stop_mode=clone:stop=2,trim=end_frame={n_frames}[vc]")
    tail = f"atrim=end={end:.4f},afade=t=out:st={max(end - 0.025, 0):.4f}:d=0.025"
    if beds:
        parts.append("[voice]" + "".join(beds) +
                     f"amix=inputs={1 + len(beds)}:duration=first:normalize=0,{tail}[ac]")
    else:
        parts.append(f"[voice]{tail}[ac]")
    return parts, inputs, end


def build(job: Path, encoder_args, reframe=None):
    sheet = json.loads((job / "transcript" / "cutsheet.json").read_text())
    if montage.is_montage(sheet):
        parts, inputs, total = build_montage(job, sheet, encoder_args, reframe)
        # Constant output rate: without it the last frame leaves the graph with no duration and
        # the mp4 edit list hides it (the picture ends a frame before the sound).
        fps = montage.output_fps(job, sheet)
        rate = ["-fps_mode", "cfr", "-r", f"{fps.numerator}/{fps.denominator}"]
        return finish(job, parts, inputs, encoder_args + rate, total, sheet.get("voice"))
    segs = sheet["segments"]
    if not segs:
        sys.exit("cutsheet has no segments")

    clips = []
    for s in segs:
        for key in ("clip", "start", "end", "text"):
            if key not in s:
                sys.exit(f"segment missing '{key}': {s}")
        if s["end"] <= s["start"]:
            sys.exit(f"segment end <= start: {s}")
        if s["clip"] not in clips:
            clips.append(s["clip"])

    inputs = []
    for c in clips:
        path = job / c
        if not path.exists():
            sys.exit(f"missing clip: {path}")
        inputs += ["-i", str(path)]
```

```
    sizes = {c: probe_size(job / c) for c in clips} if reframe else {}

    parts, labels = [], []
    for n, s in enumerate(segs):
        i = clips.index(s["clip"])
        start, end = float(s["start"]), float(s["end"])
        dur = end - start
        v = f"[{i}:v]trim=start={start:.3f}:end={end:.3f},setpts=PTS-STARTPTS"
        if reframe:
            v += "," + reframe_filter(sizes[s["clip"]], reframe, s.get("x_offset", 0) or 0)
        parts.append(v + f"[v{n}]")
        # pad to the exact length: audio that ends before its picture would shift every later segment's sound
        a = (f"[{i}:a]atrim=start={start:.3f}:end={end:.3f},asetpts=PTS-STARTPTS,"
             f"apad=whole_dur={dur:.3f},atrim=end={dur:.3f}")
        fade = float(s.get("fade_out", 0) or 0)
        if fade > 0:
            a += f",afade=t=out:st={max(dur - fade, 0):.3f}:d={fade:.3f}"
        parts.append(a + f"[a{n}]")
        labels.append(f"[v{n}][a{n}]")
    parts.append("".join(labels) + f"concat=n={len(segs)}:v=1:a=1[vc][ac]")
    return finish(job, parts, inputs, encoder_args,
                  sum(float(s["end"]) - float(s["start"]) for s in segs))


def measure_loudness(parts, inputs, pre):
    """First loudnorm pass on the audio branch alone (no video decoded), for a linear second pass.

    Returns loudnorm's measured values, or None when the audio branch isn't separable from the
    video (a plain cut sheet concats picture and sound together)."""
    audio = [p for p in parts if not re.search(r":v\]|\[v\d+\]|\[vc\]", p)]
    if not any(p.endswith("[ac]") for p in audio):
        return None
    graph = ";\n".join(audio + [f"[ac]{pre}loudnorm=I=-14:TP=-1.5:LRA=11:print_format=json[m]"])
    path = Path(tempfile.mkstemp(suffix=".txt")[1])
    r = subprocess.run(["ffmpeg", "-hide_banner", "-nostats", *inputs, *graph_args(graph, path),
                        "-map", "[m]", "-f", "null", "-"], capture_output=True, text=True)
    m = re.search(r"\{[^{}]*\"input_i\"[^{}]*\}", r.stderr)
    return json.loads(m.group(0)) if m else None


def finish(job: Path, parts, inputs, encoder_args, total, voice=None):
    """Polish the assembled [ac] once, map [vc], encode to outputs/base.mp4.

    Loudness goes to -14 LUFS with ONE steady gain (two-pass linear loudnorm) when the audio
    branch can be measured on its own. Single-pass loudnorm is dynamic: it lifts quiet stretches
    toward the target, so room noise under a silent moment gets pushed up.
    voice.denoise_db (optional, from the cut sheet): gentle afftdn noise reduction before loudness,
    e.g. 6. Keep it light; the voice must keep its body and the room must not sound gated.
    """
    voice = voice or {}
    pre = ""
    if voice.get("rnnoise"):
        # RNNoise (ffmpeg arnndn, a small speech-denoising network): measured on street and
        # busy-room audio it drops the background ~7-9 dB further than afftdn at 10 dB, with the
        # voice level unchanged. mix < 1 keeps a little of the room so it never sounds gated; the
        # low-cut takes the traffic/AC rumble without touching the voice's body.
        model = root_of(job) / "assets" / "audio-models" / voice.get("rnnoise_model", "sh.rnnn")
        pre = (f"highpass=f={float(voice.get('lowcut_hz', 85))},"
               f"arnndn=m='{model}':mix={float(voice.get('rnnoise_mix', 0.85))},")
    if voice.get("denoise_db"):  # alone, or as a light second pass after RNNoise for what it leaves
        pre += f"afftdn=nr={float(voice['denoise_db'])}:nf=-45:tn=1,"
    ln = "loudnorm=I=-14:TP=-1.5:LRA=11"
    # voice.gain_db pins the overall gain. Use it once the main dialogue (e.g. the main scene) sits at
    # -14 and other sections are balanced with per-segment gain_db: re-measuring the whole mix after
    # each trim swings the gain, because the loudest section dominates the measurement.
    mv = None if voice.get("gain_db") is not None else measure_loudness(parts, inputs, pre)
    if voice.get("gain_db") is not None:
        ln = f"volume={float(voice['gain_db']):.2f}dB"
        print(f"loudness: pinned fixed gain {float(voice['gain_db']):+.2f} dB", file=sys.stderr)
    if mv:
        # A plain fixed gain, not loudnorm's linear mode: with quiet camera audio (-44 LUFS needs
        # ~+30 dB) linear mode can't keep the true peak and silently falls back to dynamic, which
        # lifts the room tone under quiet moments again. The limiter below only catches peaks (clinks).
        gain = -14.0 - float(mv["input_i"])
        ln = f"volume={gain:.2f}dB"
        print(f"loudness: measured {mv['input_i']} LUFS, fixed gain {gain:+.2f} dB to -14", file=sys.stderr)
    if ln.startswith("volume="):
        # Gain FIRST, then denoise: RNNoise is tuned for speech at normal level. Fed raw camera
        # audio (~-45 LUFS) it starts classing a quieter section's voice as noise, so a -4 dB trim
        # came out -22 dB. At working level the trims stay linear.
        parts.append(f"[ac]{ln},{pre}alimiter=limit=0.89:level=disabled[ao]")
    else:
        parts.append(f"[ac]{pre}{ln},alimiter=limit=0.89:level=disabled[ao]")
```

```
    out = job / "outputs" / "base.mp4"
    cmd = ["ffmpeg", "-y", "-hide_banner", *inputs,
           *graph_args(";\n".join(parts), job / "outputs" / "base.filtergraph.txt"),
           "-map", "[vc]", "-map", "[ao]", "-map_chapters", "-1",
           *encoder_args, "-pix_fmt", "yuv420p",
           "-c:a", "aac", "-b:a", "256k", "-ar", "48000",
           "-movflags", "+faststart", str(out)]
    return cmd, out, total


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("job", type=Path)
    ap.add_argument("--dry-run", action="store_true", help="print the command, do not run")
    ap.add_argument("--encoder", help="e.g. h264_nvenc; overrides the platform default")
    ap.add_argument("--style", help="style name; applies its rough_cut.reframe for --format")
    ap.add_argument("--format", choices=["long", "short-explainer", "short-raw"])
    args = ap.parse_args()

    reframe = None
    if args.style:
        if not args.format:
            sys.exit("--style needs --format")
        style = json.loads((root_of(args.job) / "styles" / args.style / "style.json").read_text())
        reframe = style.get("rough_cut", {}).get("reframe", {}).get(args.format)

    enc = ["-c:v", args.encoder] if args.encoder else default_encoder()
    cmd, out, total = build(args.job, enc, reframe)
    print(f"{total:.2f}s of output from cutsheet -> {out}")
    if args.dry_run:
        print(" ".join(repr(c) if " " in c or "\n" in c else c for c in cmd))
        return
    out.parent.mkdir(parents=True, exist_ok=True)
    subprocess.run(cmd, check=True)


if __name__ == "__main__":
    main()
```

FILE: .claude/skills/rough-cut/scripts/strip.py

2,048 bytes

```
#!/usr/bin/env python3
"""Labelled frame strip from a (proxy) video for choosing shots and x_offsets.

Each frame is labelled with its source time; a cyan box marks the centred 9:16 window the
vertical reframe shows (1080 of 2592 px at 135%), so the subject can be checked against it.
Usage: strip.py <video> <out.jpg> <fps> a-b[:label] [a-b[:label] ...]
"""
import io
import subprocess
import sys

from PIL import Image, ImageDraw, ImageFont


def main():
    if len(sys.argv) < 5:
        sys.exit("usage: strip.py <video> <out.jpg> <fps> a-b[:label] ...")
    video, out, fps = sys.argv[1], sys.argv[2], float(sys.argv[3])
    font = ImageFont.truetype("/System/Library/Fonts/Supplemental/Arial Bold.ttf", 15)
    W, H = 240, 135
    tiles = []
    for win in sys.argv[4:]:
        rng, _, label = win.partition(":")
        a, b = map(float, rng.split("-"))
        t = a
        while t <= b + 1e-6:
            png = subprocess.run(["ffmpeg", "-v", "error", "-ss", f"{t:.3f}", "-i", video, "-frames:v", "1",
                                  "-vf", f"scale={W}:{H}", "-f", "image2pipe", "-vcodec", "png", "-"],
                                 capture_output=True, check=True).stdout
            im = Image.open(io.BytesIO(png)).convert("RGB")
            d = ImageDraw.Draw(im)
            bw = W * 1080 / 2592
            x0 = (W - bw) / 2
            d.rectangle([x0, 0, x0 + bw, H - 1], outline=(0, 255, 255))
            d.rectangle([0, 0, 78, 18], fill=(0, 0, 0))
            d.text((2, 1), f"{t:.2f}", fill=(255, 230, 0), font=font)
            if label and t == a:
                d.rectangle([0, H - 18, W, H], fill=(0, 0, 0))
                d.text((2, H - 18), label, fill=(255, 255, 255), font=font)
            tiles.append(im)
            t += 1 / fps
```

```
    C = 8
    R = (len(tiles) + C - 1) // C
    sheet = Image.new("RGB", (C * W, R * H))
    for i, im in enumerate(tiles):
        sheet.paste(im, ((i % C) * W, (i // C) * H))
    sheet.save(out, quality=82)
    print(out, len(tiles))


if __name__ == "__main__":
    main()
```

FILE: .claude/skills/rough-cut/scripts/transcribe.sh

2,110 bytes

```
#!/usr/bin/env bash
# Transcribe raw/ and broll/ once with WhisperX large-v3 + wav2vec2 alignment.
# Re-running skips straight to the saved transcripts.
# The WhisperX env lives in ~/.cache/video-editor/whisperx (3-5 GB, built once by uv).
set -euo pipefail

JOB="${1:?usage: transcribe.sh projects/<job>}"
ENV_DIR="${WHISPERX_ENV:-$HOME/.cache/video-editor/whisperx}"
LANG_CODE="${WHISPERX_LANG:-en}"
mkdir -p "$JOB/transcript"

if [ ! -x "$ENV_DIR/bin/whisperx" ]; then
  echo "Building WhisperX environment in $ENV_DIR (one time, several minutes)..."
  uv venv --python 3.11 "$ENV_DIR"
  uv pip install --python "$ENV_DIR/bin/python" whisperx
fi

DEVICE=cpu; COMPUTE=int8
if command -v nvidia-smi >/dev/null 2>&1; then DEVICE=cuda; COMPUTE=float16; fi

transcribe_dir() {
  local src="$1" out_json="$2"
  if [ -f "$out_json" ]; then
    echo "Already transcribed: $out_json (skipping)"; return
  fi
  shopt -s nullglob
  # Declare, then assign: macOS bash 3.2 under `set -u` loses a `local arr=(...)` array.
  local -a files
  files=("$src"/*.{mp4,mov,mkv,m4v,wav,m4a,mp3,MP4,MOV})
  if [ ${#files[@]} -eq 0 ]; then echo "No media in $src"; return; fi
  local tmp; tmp="$(mktemp -d)"
  for f in "${files[@]}"; do
    "$ENV_DIR/bin/whisperx" "$f" --model large-v3 --language "$LANG_CODE" \
      --device "$DEVICE" --compute_type "$COMPUTE" \
      --output_format json --output_dir "$tmp"
  done
  # Merge into {"clips": {"raw/A001.mp4": <whisperx json>, ...}}
  python3 - "$src" "$tmp" "$out_json" "${files[@]}" <<'PY'
import json, os, sys
src, tmp, out, *files = sys.argv[1:]
job_rel = os.path.basename(os.path.normpath(src))
clips = {}
for f in files:
    stem = os.path.splitext(os.path.basename(f))[0]
    with open(os.path.join(tmp, stem + ".json")) as fh:
        clips[f"{job_rel}/{os.path.basename(f)}"] = json.load(fh)
with open(out, "w") as fh:
    json.dump({"clips": clips}, fh, indent=1)
print(f"Wrote {out} ({len(clips)} clips)")
PY
  rm -rf "$tmp"
}

transcribe_dir "$JOB/raw" "$JOB/transcript/transcript.json"
[ -d "$JOB/broll" ] && transcribe_dir "$JOB/broll" "$JOB/transcript/broll.json"
```

FILE: .claude/skills/rough-cut/scripts/unknown_words.py

2,672 bytes

```
#!/usr/bin/env python3
"""Second mishear pass: print transcript words that are neither ordinary English nor known.

Compares every word against a system dictionary plus the brand.md and per-video
mishear lists. Judge each hit in context: recurring -> brand.md, one-off ->
transcript/mishears.json.
```

```
--long-spans also lists words longer than 1.2 s: WhisperX sometimes merges a stumble
and its retake into one span. Run silencedetect across those before cutting.
"""
import argparse
import json
import re
import sys
from collections import Counter
from pathlib import Path

sys.path.insert(0, str(Path(__file__).parent))
from remap import clip_words, load_mishears  # noqa: E402

DICTS = ["/usr/share/dict/words", "/usr/share/dict/american-english", "/usr/share/dict/british-english"]


def load_dictionary():
    words = set()
    for d in DICTS:
        p = Path(d)
        if p.exists():
            words |= {w.strip().lower() for w in p.read_text(errors="ignore").splitlines()}
    if not words:
        sys.exit("No system dictionary found. macOS ships /usr/share/dict/words; "
                 "on Debian/Ubuntu: sudo apt install wamerican")
    return words


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("job", type=Path)
    ap.add_argument("--long-spans", action="store_true")
    args = ap.parse_args()

    dictionary = load_dictionary()
    fixes = load_mishears(args.job)
    known = dictionary | set(fixes) | {v.lower() for v in fixes.values()}

    hits, contexts = Counter(), {}
    for name in ("transcript.json", "broll.json"):
        path = args.job / "transcript" / name
        if not path.exists():
            continue
        for clip, data in json.loads(path.read_text())["clips"].items():
            words = list(clip_words(data))
            for i, w in enumerate(words):
                core = re.sub(r"[^\w'-]", "", w["word"]).lower().strip("'-")
                if args.long_spans and float(w["end"]) - float(w["start"]) > 1.2:
                    print(f"LONG  {clip} {w['start']:.2f}-{w['end']:.2f} '{w['word']}'")
                if not core or core.isdigit():
                    continue
                base = re.sub(r"('s|s|es|ed|ing)$", "", core)
                if core in known or base in known:
                    continue
                hits[core] += 1
                ctx = " ".join(x["word"] for x in words[max(i - 5, 0): i + 6])
                contexts.setdefault(core, f"{clip} @{w['start']:.1f}s: ...{ctx}...")

    for word, n in hits.most_common():
        print(f"{n:3d}x  {word:<20} {contexts[word]}")
    if not hits:
        print("No unknown words.")


if __name__ == "__main__":
    main()
```

FILE: .claude/skills/rough-cut/scripts/words.py

1,278 bytes

```
#!/usr/bin/env python3
"""Print source word timings, and the gap before each word, for windows of a job's transcript.

The gaps are what montage and fragment cuts are chosen from: cut in a gap, keep short natural
pauses, and lift a fragment whole from pause to pause.
Usage: words.py projects/<job> a-b [a-b ...]      (seconds in the source clip)
"""
import json
import sys
from pathlib import Path


def main():
```

```
    if len(sys.argv) < 3:
        sys.exit("usage: words.py projects/<job> a-b [a-b ...]")
    job = Path(sys.argv[1])
    clips = json.loads((job / "transcript" / "transcript.json").read_text())["clips"]
    for name, clip in clips.items():
        words = [w for s in clip["segments"] for w in s["words"] if "start" in w]
        for win in sys.argv[2:]:
            a, b = map(float, win.split("-"))
            prev_end, row = None, []
            for w in words:
                if w["end"] > a and w["start"] < b:
                    gap = "" if prev_end is None else f"(+{w['start'] - prev_end:.2f})"
                    row.append(f"{gap}{w['word']}[{w['start']:.2f}-{w['end']:.2f}]")
                if w["start"] < b:
                    prev_end = w["end"]
            print(f"--- {name} {win}\n" + " ".join(row))


if __name__ == "__main__":
    main()
```

Skill: graphics

FILE: .claude/skills/graphics/SKILL.md

13,463 bytes

```
---
name: graphics
description: Stage 2. Decide which lines of the rough cut earn a graphic, write a validated graphics plan, then bui
» ld every graphic as HyperFrames code (HTML/CSS/GSAP) and composite it over the base cut with FFmpeg. Use after the
» rough cut, or whenever the user asks to add, change or fix a graphic, card, takeover, zoom, stat or screenshot in a
»  video.
---

# Graphics

Its whole job: decide which lines earn a graphic, then build each one.

Two halves that stay separate: **plan, then build**. Planning is a judgment problem,
building is an engineering problem. Mixing them produces a plan written to be easy to
build, which is the wrong optimisation. Finish and validate the plan before writing any
composition.

Read first: `brand.md`, `styles/<style>/style.md` + `style.json`. If a taste skill is
installed, apply it: every graphic is front-end design.

---

## Part 1: the plan

### Read three things before deciding a single beat

1. `outputs/transcript.cut.json` from the rough cut.
2. Comments in the user's script document. For a .docx, pull them from the document XML
   (`word/comments.xml`, anchored via `commentRangeStart` in `word/document.xml`), not a
   library that drops them. For Google Docs, read comments through the Drive API/connector.
3. The B-roll narration in `transcript/broll.json`.

### For every beat, answer in order

**1. Does this line need a graphic? Default to no.** A graphic on every line is wrong.
Plain beats give rhythm and let the face carry the moment. A beat earns a graphic when:
- it is the hook;
- it names something concrete and showable (a stat, a screenshot, a before and after);
- it is a payoff worth emphasising;
- it describes a process a picture would make instantly clearer;
- the user explicitly asked for one.

Leave connective tissue, transitions, asides and emotional delivery plain.

**2. What kind?** Think in rich terms first, then flatten to exactly one of seven kinds:
`stat`, `card`, `screenshot`, `takeover`, `zoom`, `diagram`, `broll` (slot filled by the
`broll` skill). Reach for showing over telling: screen recordings, screenshots, diagrams
and stats beat text cards almost every time. Reserve cards for the hook and punchlines,
and even then give them motion and a visual element, not a wall of type.

**3. Where?**
| Format           | Placement                                                          |
|------------------|--------------------------------------------------------------------|
| Short explainer  | Graphics in the top half, face in the bottom half.                 |
| Short raw        | Whatever the style file allows, nothing more (often one hook card or none; never a title card
» the style does not ask for). |
| Long form        | Full-frame takeovers, lower thirds, no reframe.                    |

**4. What exactly?** Write the actual creative direction: what is on screen, the
hierarchy, the hero element, and critically **what animates and in what order**.
Concrete enough that the build step is not guessing.

### Write the plan

`transcript/graphics-plan.json`, plus a human-readable table shown to the user for review:

```json
{
  "format": "long",
  "style": "editorial",
  "beats": [
    {"id": "g01", "start": 0.00, "end": 3.40, "kind": "card",
     "direction": "Hook card. Eyebrow 'THE $90 EDITOR' clip-reveals, then title word by word...",
     "notes": "script comment: 'start zoomed in and rapidly zoom out'"}
  ]
}
```
```

Beats with no graphic are simply not in the list.

Validate before the build ever sees it

`scripts/validate_plan.py projects/<job>` checks: required fields present, kind is one of the seven, start < end, sorted ascending, no overlaps, and **consecutive beats either abut exactly or leave a gap of more than 1 s**. A gap of a few tenths flashes raw footage for a fraction of a second in the composite and almost always means the plan meant to abut.

Safe zones

Format	Frame	Keep key visuals inside
Short form	1080x1920	y 200 to 1620. Top 200 px and bottom 300 px are background only.
Long form	1920x1080	Title safe, 10% margins. Outro: keep the right 40% clear for end-screen cards.

The short-form bands are not a guideline. The platform UI, username, audio tag and progress bar all land there.

Part 2: the build

Generate, do not hand-write

One build script in `projects/<job>/graphics-build/` (e.g. `build.mjs`) holds the shared CSS, the per-graphic markup and the per-graphic animation, and emits one composition per part plus `render.sh` and `assemble.sh`. That folder is the real progress on the job, so it lives in the project, never in a temp directory. Only heavy regenerable renders go to `graphics-build/renders/` (git-ignored).

Pin HyperFrames in `graphics-build/package.json` to the version `doctor` passed on. Never upgrade it silently.

Classify every part: does it change the footage underneath, or float on top?

- **Overlay**: a card, panel, callout. Renders standalone to a transparent file (ProRes 4444 / VP9 alpha) and composites over the base at its timestamp.
- **Segment**: a takeover, full-screen cutaway, anything that replaces the frame. Renders with its own slice of the base baked in, as an opaque file covering the base for its window.

The base rough cut is never re-rendered. Editing one graphic = regenerate one composition, re-render one part in seconds, run one FFmpeg composite pass. Lock parts one at a time.

Non-negotiables

- **Graphics hold until the next part starts.** No early fade leaving dead air.
- **The PiP enters once per graphics run.** Chain everything between entries. Bouncing full frame -> PiP -> full frame is the single most amateur-looking thing an AI editor does. Hard cuts between card contents are fine; a full-screen bounce never is.
- **Continuous motion on any beat of 20 s or longer.** Write out explicitly what is moving across its entire duration, not just the entrance. A count-up done at 6 s of a 19 s beat reads as a frozen frame for 13 s.
- **Real assets over recreations.** The actual logo, screenshot, chart, with a slow pan or push. Never a redrawn approximation. If an icon or illustration cannot be built in HTML and does not exist, generate it mid-edit with the Higgsfield MCP and composite it in.
- **Measure, do not estimate.** Pull the actual frame, measure the element's bounding box in pixels, set the zoom from that measurement.
- **Check the tail of every clip, not just the start.** A retake seam leaves a bad frame at the out point and nobody watches the last second.
- **Assets outlast their window.** Every part gets ~0.5 s of tail margin past its nominal end.
- **Local direction never silently overrides a style convention.** A script comment that conflicts with the style file is a conflict to flag to the user, not a decision to make quietly. Following the more recent instruction is recency bias, not judgment.

HyperFrames: the timeline contract

Compositions are **seeked, not played**. The renderer jumps to arbitrary frames.

- One master timeline, created paused, every tween at an **absolute second**. No relative offsets.
- **Never random values.** The same frame must render identically on every seek.
- **Never real-time timers.** No `setTimeout`, no `requestAnimationFrame` loops. Everything comes off timeline position.
- Every exit landing on a boundary needs an **explicit hard kill**: opacity set to 0 at that exact time. An unresolved tween pops instead of finishing.
- Time is always in **seconds**, never frames.
- Every entrance is a **`fromTo`**, never a bare `to` (no defined start state when seeked into).
- Never put a CSS transform and a GSAP tween on the same property.

Things that silently do not render (nothing errors)

- **Never transform a `<video>` directly.** Scale/move on a video gets composited away in the headless render; the face just vanishes. Reframe through layout: wrap the video in a `div` with `overflow: hidden` and animate the wrapper's `left/top/width/height`.
- **CSS `blur()` filters are not render-safe.** For a blur-in text reveal use a per-letter opacity stagger, ~0.045 s between letters.
- **Grayscale filters fail the same way.** Fake desaturation by tweening colour toward a flatter value.
- **Class-name tweens do not survive the seek** and can wipe the base class styling. Tween the actual CSS properties.
- **Near-zero-duration tweens are unreliable.** Give every instant state change 0.2-0.35 s. It still reads as a cut.
- **Raw emoji glyphs hang the render** at full CPU, forever, no error. Use the brand font or a pre-rendered transparent PNG. Tell: a render still going at 3x the length of a same-sized part is an emoji, not a real hang.
- **Transparent overlays have nothing behind them, so `backdrop-filter` does nothing.** Design frosted panels to read on their own fill.

Linter and workflow

- `lint` and `validate` are the gate. Run both on every part before rendering.
- Contrast warnings on deliberately dim text: raise opacity, not brightness.
- A dense-track warning on a short build is normal. Do not refactor into sub-compositions to silence it.

The composite traps (FFmpeg, show up only at assembly)

- **Match every part to the base frame rate exactly**, probed with ffprobe (`24000/1001`, not `24`). Mixed rates drift.
- **Segments carry their own base slice**, cut once from the base, so first and last frames match at the seam.
- **Cut that slice at the time it is placed, not the time it was built.** If the base is re-spliced and graphics shift, re-cut and re-render every segment. Overlays just slide.
- **Every overlay gets `eof_action=pass`, not repeat.** Chaining short overlays over a long base otherwise makes the scheduler duplicate output frames on a periodic cadence (measured 1 in 4): constant frame rate on paper, ~18 real frames/s in a 24 fps costume. Visible judder; every input measures clean alone. The one exception is a deliberately held frame (e.g. an outro push-in that must not zoom back out): that one gets `repeat`.
  ```
  [0:v][1:v]overlay=enable='gte(t,12.4)*lt(t,17.9)':eof_action=pass[v1]; ...
  ```
  (Offset each overlay input with `-itsoffset <start>` so its t=0 lands at its window.)
- **Gate overlays with a half-open window, `gte(t,a)*lt(t,b)`, never `between(t,a,b)`.** `between` includes both ends, so where one overlay abuts the next, the frame at the boundary draws both: two overlays stacked on top of each other for one frame.
- **`assemble.sh` must run `scripts/dupe_check.py` on the output and fail above 8%.** Clean is under 3%, the bug is ~25%. Do not mask it by forcing a frame rate; the output is already constant.
- **Never use frame padding (`tpad`) to hold a zoom.** It never reaches EOF and can balloon a few-second clip into gigabytes.
- **Browser segments darken the footage ~3% luma**, so the face dips at every seam. Fix at the root, in order:
  1. Do footage motion in FFmpeg, not the browser. A zoom or push-in is pure geometry. Only use a browser segment when live graphics must reveal behind the moving face.
  2. If it must be a browser segment, render its source frames as PNG (halves the dip), then close the rest with a gamma correction at assemble time (`eq=gamma=1.03` or measured), not a flat gain, so black and white stay pinned and highlights do not clip.
- **An FFmpeg zoom on a mid-video slice needs its frame counter reset**, or it comes out constant and reads as a hard cut. Build the slice with `trim=...,setpts=PTS-STARTPTS` inside the filtergraph, not with `-ss`. An intro zoom works by luck at t=0; later ones silently do not.
- **Anchor footage motion to measured scene cuts, not nominal ones.** The rendered base drifts from the cut sheet by a few hundred ms by the end of a reel. Detect the real cut: `ffmpeg -i base.mp4 -vf "select='gt(scene,0.3)',showinfo" -f null -`.
- **Screen recordings carry black bars.** Run `cropdetect` before compositing one into a card. And size the card bigger than feels right: numbers that look generous in CSS render small.

Review (with the `watch` skill, in sub-agents)

- **Early spot review** after the first ~10% of parts are rendered and composited. A wrong placement habit caught once is a fix; caught at the end it is a rebuild.
- **Final review, two passes:** technical QA as a checklist, then composition as its own named step (every overlay against the style's rules for its category; every visual moment in the direction built or explicitly flagged as skipped).
- Inspect every seam and every part's last second.

Scripts

- `scripts/validate_plan.py <job>`: gate before any build.
- `scripts/dupe_check.py <video> [--max 8]`: duplicate-frame ratio (near-identical, since lossy encodes break byte-exact matches; exact count reported too); exit 1 above max.

macOS: filtergraphs

```
- **Never pass a long filtergraph inline on macOS.** XProtect blocks ffmpeg launched with a
  long multi-line `-filter_complex` argument as "Malicious Script Blocked" and kills the
  terminal (and Claude Code) with it. Write the graph to a file and pass it with
  `-/filter_complex <file>` (ffmpeg 7+); `splice.py` and `mix.py` do this via `graph_args()`.
  Applies to any ad-hoc ffmpeg command too, including composites.
```

FILE: .claude/skills/graphics/scripts/dupe_check.py

2,361 bytes

```
#!/usr/bin/env python3
"""Count duplicate consecutive video frames in a render.

Catches the overlay eof_action=repeat judder: output is constant frame rate on paper,
but ~1 in 4 frames is a copy of the one before. Clean < 3%, bug ~25%.
Exit 1 when the duplicate ratio exceeds --max (default 8%).

Duplicates are only byte-identical in a lossless render. After a lossy encode, x264
re-codes each copy with small residuals, so this uses mpdecimate with tight thresholds
("near-identical") as the verdict and reports the exact-hash count alongside.

A deliberately held frame (e.g. an outro push-in that holds) adds legitimate
duplicates; pass --ignore-after <seconds> to exclude that tail.
"""
import argparse
import re
import subprocess
import sys


def ffmpeg(args, trim):
    cmd = ["ffmpeg", "-hide_banner", "-nostats", "-i", args.video]
    if trim:
        cmd += ["-t", str(trim)]
    return cmd


def exact_dupes(args):
    out = subprocess.run(ffmpeg(args, args.ignore_after) +
                         ["-map", "0:v:0", "-f", "framemd5", "-"],
                         capture_output=True, text=True, check=True).stdout
    hashes = [ln.rsplit(",", 1)[-1].strip() for ln in out.splitlines() if ln and not ln.startswith("#")]
    return len(hashes), sum(a == b for a, b in zip(hashes, hashes[1:]))


def near_kept(args):
    vf = "mpdecimate=hi=256:lo=128:frac=0.33:max=0,showinfo"
    err = subprocess.run(ffmpeg(args, args.ignore_after) + ["-map", "0:v:0", "-vf", vf, "-f", "null", "-"],
                         capture_output=True, text=True, check=True).stderr
    return len(re.findall(r"Parsed_showinfo.*\bn:\s*\d+", err))


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("video")
    ap.add_argument("--max", type=float, default=8.0, help="max duplicate %% (default 8)")
    ap.add_argument("--ignore-after", type=float, help="ignore frames after this time (s)")
    args = ap.parse_args()

    total, exact = exact_dupes(args)
    if total < 2:
        sys.exit("not enough frames to check")
    near = total - near_kept(args)
    pct = 100.0 * near / (total - 1)
    verdict = "FAIL" if pct > args.max else "ok"
    print(f"{verdict}: {near}/{total - 1} duplicate frames ({pct:.1f}%, max {args.max}%); "
          f"byte-identical: {exact}")
    sys.exit(1 if pct > args.max else 0)


if __name__ == "__main__":
    main()
```

FILE: .claude/skills/graphics/scripts/validate_plan.py

2,579 bytes

```
#!/usr/bin/env python3
"""Validate transcript/graphics-plan.json before the build step sees it. Exit 1 on any error."""
import json
import sys
from pathlib import Path
```

```
KINDS = {"stat", "card", "screenshot", "takeover", "zoom", "diagram", "broll"}
REQUIRED = ("id", "start", "end", "kind", "direction")
EPS = 0.001  # "abut exactly" tolerance, one millisecond


def validate(plan: dict, duration: float | None = None) -> list[str]:
    errors = []
    beats = plan.get("beats")
    if not isinstance(beats, list):
        return ["plan has no 'beats' list"]
    ids = set()
    for i, b in enumerate(beats):
        tag = b.get("id", f"#{i}")
        missing = [k for k in REQUIRED if b.get(k) in (None, "")]
        if missing:
            errors.append(f"{tag}: missing {', '.join(missing)}")
            continue
        if b["id"] in ids:
            errors.append(f"{tag}: duplicate id")
        ids.add(b["id"])
        if b["kind"] not in KINDS:
            errors.append(f"{tag}: kind '{b['kind']}' not in {sorted(KINDS)}")
        if not float(b["start"]) < float(b["end"]):
            errors.append(f"{tag}: start {b['start']} is not before end {b['end']}")
        if duration is not None and float(b["end"]) > duration + EPS:
            errors.append(f"{tag}: ends at {b['end']} past the cut's duration {duration}")

    good = [b for b in beats if all(b.get(k) not in (None, "") for k in REQUIRED)]
    for prev, cur in zip(good, good[1:]):
        p, c = prev["id"], cur["id"]
        if float(cur["start"]) < float(prev["start"]):
            errors.append(f"{c}: not sorted ascending (starts before {p})")
            continue
        gap = float(cur["start"]) - float(prev["end"])
        if gap < -EPS:
            errors.append(f"{p} -> {c}: overlap of {-gap:.3f}s")
        elif EPS < gap <= 1.0:
            errors.append(f"{p} -> {c}: gap of {gap:.3f}s. Abut exactly or leave more than 1s "
                          "(a short gap flashes raw footage)")
    return errors


def main():
    if len(sys.argv) != 2:
        sys.exit("usage: validate_plan.py projects/<job>")
    job = Path(sys.argv[1])
    plan = json.loads((job / "transcript" / "graphics-plan.json").read_text())
    cut = job / "outputs" / "transcript.cut.json"
    duration = json.loads(cut.read_text()).get("duration") if cut.exists() else None
    errors = validate(plan, duration)
    for e in errors:
        print("ERROR", e)
    n = len(plan.get("beats") or [])
    print(f"{n} beats, {len(errors)} errors")
    sys.exit(1 if errors else 0)


if __name__ == "__main__":
    main()
```

Skill: broll

FILE: .claude/skills/broll/SKILL.md

4,201 bytes

```
---
name: broll
description: Stage 3. Turn a beat that has no footage into a rendered clip, either a HyperFrames motion graphic fro
» m one of eight scene templates or AI footage generated through the Higgsfield MCP, then write a manifest entry. Doe
» s not pick windows in existing footage and does not composite. Use for the "broll" slots in the graphics plan or wh
» en the user asks for generated B-roll.
---

# AI B-roll

Its whole job: turn a beat that has no footage into a rendered clip.

What it does **not** do:
- Pick windows in footage the user already shot. That belongs to `graphics`.
- Composite anything into the final video. That also belongs to `graphics`.

It generates a clip and writes a manifest entry, and that is it. Keeping generation separate
from placement is what makes both debuggable.

Inputs: `broll` beats from `transcript/graphics-plan.json`, `brand.md`, the style file.
Outputs: `projects/<job>/graphics-build/broll/<beat-id>.mov|mp4` and an entry in
`projects/<job>/graphics-build/broll/manifest.json`.

## Pick the scene template first

Map every slot to one of eight templates **before** writing a prompt. Picking the right
one up front is most of what stops the output reading generic.

| The beat is about          | Reach for        |
|----------------------------|------------------|
| One big number landing     | Stat reveal      |
| Several categories at once | Data breakdown   |
| Parts of a whole           | Pie or donut     |
| Things ranked              | Podium           |
| A system or process        | Flowchart        |
| "Look what showed up"      | Phone notification |
| Old versus new             | Before and after |
| Words as the payoff        | Kinetic type     |

Build data-shaped templates in HyperFrames (it follows every rule in the `graphics` skill's
HyperFrames section). Use Higgsfield for footage and imagery HTML cannot draw.
Higgsfield costs a few dollars per generated segment: tell the user the count before
generating more than three.

## The twelve design rules

These separate motion design from PowerPoint. **Make at least six of them explicit in every
single prompt or composition brief** rather than hoping the render figures them out.

1. Text never just fades in. Clip-mask reveals, or word by word.
2. Nothing animates simultaneously. Stagger everything by at least 0.4 s.
3. The background is never flat. Gradient, vignette, or a fine grid.
4. The accent colour appears on exactly one element per scene.
5. Numbers count up or flip. They never just appear.
6. Exits are designed. Elements leave with purpose.
7. Generous whitespace. More than feels right.
8. One focal point per frame. Never more than two things moving.
9. Scale is dramatic. Primary numbers 160 px minimum.
10. Connectors and dividers draw in, never appear.
11. Small premium details. Thin highlights, low-opacity reflections.
12. Motion blur on fast travel, removed once settled.

**The failure list** (check every brief against it): the same layout reused for every beat;
"fade in" instead of a real entrance; no stagger; a flat background; a vague ease like
"smooth" instead of a named one (`power3.out`, `expo.inOut`); small type; no exits; too many
colours; everything moving at once.

## Technical rules

- **Render at the base video's exact frame rate, probed, never guessed.** If the base is
  23.976, pass `24000/1001`, not `24`. `scripts/finalize.py` probes it for you.
- **Every generated clip is silent.** The base voice keeps playing underneath. If a render
  produces an audio stream anyway, strip it before writing the manifest.
- **Duration = beat window + 0.5 s tail margin.** A clip that ends exactly on its boundary
  is a bug waiting for the next composite.
- Review each clip with the `watch` skill (sub-agent) before it goes in the manifest: both
  the first and last second.
```

```
## Scripts

`scripts/finalize.py <job> <beat-id> <clip>`: probes the base frame rate, conforms the clip
to it, strips audio, checks duration >= window + 0.5 s, writes it to
`graphics-build/broll/<beat-id>.mp4` and upserts the manifest entry
`{id, file, start, end, duration, fps, template, source}`.
Pass `--template` and `--source higgsfield|hyperframes`.
```

FILE: .claude/skills/broll/scripts/finalize.py

3,097 bytes

```
#!/usr/bin/env python3
"""Conform a generated B-roll clip to the base cut and record it in the manifest.

- frame rate: probed from outputs/base.mp4, passed as an exact rational (24000/1001)
- audio: stripped, always
- duration: must be >= beat window + 0.5 s tail margin
"""
import argparse
import json
import subprocess
import sys
from pathlib import Path

TAIL = 0.5
TEMPLATES = {"stat-reveal", "data-breakdown", "pie", "podium", "flowchart",
             "phone-notification", "before-after", "kinetic-type", "footage"}


def probe(path: Path, entries: str, stream="v:0") -> str:
    return subprocess.run(
        ["ffprobe", "-v", "error", "-select_streams", stream, "-show_entries", entries,
         "-of", "default=nw=1:nk=1", str(path)],
        capture_output=True, text=True, check=True).stdout.strip()


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("job", type=Path)
    ap.add_argument("beat_id")
    ap.add_argument("clip", type=Path)
    ap.add_argument("--template", required=True, choices=sorted(TEMPLATES))
    ap.add_argument("--source", required=True, choices=["higgsfield", "hyperframes"])
    args = ap.parse_args()

    plan = json.loads((args.job / "transcript" / "graphics-plan.json").read_text())
    beat = next((b for b in plan["beats"] if b["id"] == args.beat_id), None)
    if not beat:
        sys.exit(f"beat {args.beat_id} not in graphics-plan.json")
    if beat["kind"] != "broll":
        sys.exit(f"beat {args.beat_id} is kind '{beat['kind']}', not broll")

    need = float(beat["end"]) - float(beat["start"]) + TAIL
    fps = probe(args.job / "outputs" / "base.mp4", "stream=r_frame_rate")
    have = float(probe(args.clip, "format=duration", stream="v:0") or 0)
    if have + 1e-3 < need:
        sys.exit(f"clip is {have:.2f}s, needs >= {need:.2f}s (window + {TAIL}s tail). Regenerate longer.")

    out_dir = args.job / "graphics-build" / "broll"
    out_dir.mkdir(parents=True, exist_ok=True)
    out = out_dir / f"{args.beat_id}.mp4"
    subprocess.run(
        ["ffmpeg", "-y", "-hide_banner", "-loglevel", "error", "-i", str(args.clip),
         "-t", f"{need:.3f}", "-an", "-vf", f"fps={fps}", "-r", fps,
         "-c:v", "libx264", "-crf", "16", "-preset", "medium", "-pix_fmt", "yuv420p", str(out)],
        check=True)
    if probe(out, "stream=index", stream="a"):
        sys.exit(f"{out} still has audio. Refusing to write manifest.")

    manifest_path = out_dir / "manifest.json"
    manifest = json.loads(manifest_path.read_text()) if manifest_path.exists() else {"clips": []}
    manifest["clips"] = [c for c in manifest["clips"] if c["id"] != args.beat_id]
    manifest["clips"].append({
        "id": args.beat_id, "file": out.name,
        "start": beat["start"], "end": beat["end"], "duration": round(need, 3),
        "fps": fps, "template": args.template, "source": args.source,
    })
    manifest["clips"].sort(key=lambda c: c["start"])
    manifest_path.write_text(json.dumps(manifest, indent=1))
    print(f"{out} @ {fps} fps, {need:.2f}s, silent. Manifest updated.")
```

```
if __name__ == "__main__":
    main()
```

Skill: finishing

FILE: .claude/skills/finishing/SKILL.md

9,321 bytes

```
---
name: finishing
description: Stage 4. The finishing pass on a graphics-composited cut, in fixed order - burned-in captions from the
» remapped transcript (optional, off for long form), a flat licensed music bed, then a handful of real sound effects
» - followed by review. Use when the user asks for captions, music, sound effects, or to finish/polish a video.
---

# Finishing pass

Its whole job: captions, a music bed, and a few sound effects, then review.

Three sub-steps. Each can run alone, but when run together they **always run in this order**,
because each operates on whatever the previous one produced:

1. Captions -> `outputs/captioned.mp4`
2. Music -> `outputs/music.mp4`
3. SFX -> `outputs/sfx.mp4`

Each script picks its own input: the newest of the prior-stage files that exist
(`graphics.mp4` -> `base.mp4` for captions; `captioned.mp4` -> `graphics.mp4` -> `base.mp4`
for music; and so on). Do not hand them an input path.

## 1. Captions

- Burned in from `outputs/transcript.cut.json`, styled from `style.json` `captions.<format>`
  and the caption font in `brand.md` (optional `shadow_px` / `shadow_color` / `shadow_opacity`
  for a drop shadow, `uppercase`). Words are grouped per the style's `words_per_line`,
  breaking early at sentence punctuation. Highlighting follows the style's `highlight`:
  `karaoke` colours each word as spoken; `keywords` colours only the words you choose, listed
  as word indices into `transcript.cut.json` in `transcript/caption-emphasis.json`
  (`{"indices": [12, 57]}`); `none` colours nothing. For `keywords`, pick load-bearing words
  per the caption voice in `brand.md` before running the script.
- **Never caption a file that is already captioned.** `captions.py` refuses any input whose
  name or metadata says captioned.
- **Genuinely optional.** Long form skips captions entirely: YouTube serves its own and
  burn-ins clutter a 16:9 frame. If the style says `enabled: false`, the script exits
  without writing anything and later steps fall through to the graphics/base file.
- Short form: caption block must sit inside the safe zone (y 200-1620) and not cover the face
  or a graphic. Check with `watch`.
- **Lines are chosen as phrases**, not filled greedily: a caption never spans a sentence end, a
  pause over 0.4 s or hidden words; breaks prefer punctuation and the end of an audio piece;
  a line never ends on "the", "of", "a"... and never runs wider than the frame minus 160 px.
- **Not captioned:** cut-sheet segments whose text is bracketed (`[SOUND: ...]`: sound, not
  speech), and any window where the graphics plan has `"hide_captions": true` (a big
  title card or text already showing those words).
- **No libass?** `captions.py` checks for ffmpeg's `subtitles` filter. Without it (Homebrew's
  default ffmpeg has none) it draws each line with PIL from the brand font file and overlays
  it; same size, colour, shadow and baseline. Karaoke highlighting needs libass.
- Run: `scripts/captions.py projects/<job> --format short-explainer --style <style>`

## 2. Music

- A **flat bed at about -18 dB** (or the style's `music_db`). No ducking. No fade in. A short
  fade out on the tail (`"fade_out": 0` is a hard cut, for Shorts that loop).
- **When the style gives `music_under_voice_lu`, set the level by measurement, not by the
  number:** render the bed alone (the `[bed]` chain from `audio-filtergraph.txt`) and the
  voice alone, measure integrated LUFS per section, and set `db` so the body sits that many
  LU under the voice. The same gain is 7 dB apart on two different tracks.
- Every pass ends on a limiter (`alimiter`, -1.9 dBFS, `level=disabled`), so a bed on top of a
  speech track that already peaks at -1.5 dBTP does not push it over.

### Music edits and cutting on the beat (when the style asks for them)

- **Beat grid:** `npx hyperframes@<pinned> beats <dir>` on a scratch project whose `<audio>`
  has `data-timeline-role="music"` writes `beats/<track>.json` (a quantised grid; its bpm is
  often the eighth-note rate). Find which eighths carry the beat by low-passed energy.
- **Pieces:** `"pieces": [{"video_t", "track_t", "db_offset"?}, ...]` lays stretches of the
  track at set video times with a short crossfade (`xfade_ms`). Use it to stitch the track's
  own ending under the last line: find the ending's hits (onset scan of the last bars) and
  place the join in a break or on a beat, so the final hit lands on the last word.
- **Cut on the beat:** in a montage cut the picture is independent of the audio, so move each
  picture cut in `cutsheet.json` onto the grid (up to ~0.3 s) and re-splice, re-time the
  graphics plan, rebuild graphics and captions, then mix. Check every shot's new source window
  against camera cuts (scene score on a proxy) and keep sync shots inside their line: a cut
  that cannot move stays off-grid and is named to the user.
- The track is **user-supplied and licensed**, in `projects/<job>/audio/`. Never pick or download
  one on your own. When the user asks for suggestions, search their licensed music library
```

  (if one is connected), shortlist with reasons and preview links, and download only the track the user picks,
  into `audio/`. `scripts/track_profile.py` compares candidates without listening: a large
  loudness range reads as theatrical under a voice; check the ending is a clean close.
  If there is no track, ask the user; do not skip silently.
- **Prefer the track's own ending, unstitched:** start the track at the beat that makes its real
  final chord land on the last word or action (count back from the ending on the grid from
  `scripts/beat_grid.py`); a stitch is the fallback. The music must reach silence before the loop,
  never cut off mid-ring.
- Ducking (`--duck`) and fade-in (`--fade-in`) exist as opt-in flags. Almost never reach
  for them. If the bed is not audible enough, change the level; do not add a sidechain.

Music arc (when the style has `music_arc`)

The build and release are in the track itself; the pipeline only lines them up.

1. Pick the video's release moment from the transcript per the style (e.g. the turn from
   problem to answer), at a word onset or a scene cut measured on the rendered file.
2. Find the track's release: `scripts/find_release.py audio/<track>` ranks moments where the
   music opens up. It measures loudness only, so confirm the pick (a release can be a change
   of harmony or texture) and tell the user which moment you chose and why.
3. Add `release` to the music entry in the audio plan:
   ```json
   "music": {"file": "audio/bed.mp3", "db": -20, "fade_out": 2,
             "release": {"video_t": 9.4, "track_t": 31.2, "pre_db": -17, "ramp_s": 1.0}}
   ```
   `mix.py` trims the track's head (or delays its start) so `track_t` lands on `video_t`, holds
   `pre_db` before the release and eases to `db` over `ramp_s`.
4. If the track's build is longer than the video's intro, the head is trimmed; if it is
   shorter, the music starts late. Either is fine; say which in the review.

3. Sound effects

- **Sparse.** A handful of moments per video, not a hit on every cut.
- **Real sample files** at around -10 dB, from `assets/sfx/` (the growing library) or
  `projects/<job>/audio/`. **Never a synthesised tone.** A generated sine wave is instantly
  recognisable as not-a-sound-effect. If there are no samples, skip the step rather than
  fabricate one, and tell the user.
- When a video uses a good new sample, copy it into `assets/sfx/` so the library grows.

Pure audio passes

- **Never pass a long filtergraph inline on macOS.** XProtect blocks ffmpeg launched with a
  long multi-line `-filter_complex` argument as "Malicious Script Blocked" and kills the
  terminal (and Claude Code) with it. Write the graph to a file and pass it with
  `-/filter_complex <file>` (ffmpeg 7+); `splice.py` and `mix.py` do this via `graph_args()`.
  Applies to any ad-hoc ffmpeg command too, including composites.

Music and SFX **copy the video stream (`-c:v copy`) and never re-encode it.**

Plan on disk

Write the audio plan to `projects/<job>/transcript/audio-plan.json`:

```json
{
  "music": {"file": "audio/bed.mp3", "db": -18, "fade_out": 1.5, "duck": false, "fade_in": false},
  "sfx": [
    {"t": 3.40, "file": "assets/sfx/whoosh-soft.wav", "db": -10, "why": "hook card lands"}
  ]
}
```

`mix.py` also writes the exact filtergraph it ran to `transcript/audio-filtergraph.txt`.
When a graphics tweak later re-renders the base, re-apply the same plan
(`scripts/mix.py projects/<job>`) instead of re-deciding every placement. SFX times are on
the edited timeline, anchored to measured moments (scene cuts, word onsets), not guesses.

Review

Then review the finished file with the `watch` skill in sub-agents (technical QA checklist,
then composition). Listen-equivalents: check `ffmpeg -af ebur128` integrated loudness is
around -14 LUFS and the true peak stays below -1 dBTP.

Scripts

- `scripts/captions.py <job> --format <long|short-explainer|short-raw> [--style editorial]`
- `scripts/mix.py <job> [--music-only | --sfx-only] [--dry-run]`
- `scripts/find_release.py <track> [--top 5]`
- `scripts/beat_grid.py <track> --bpm N [--json out]`: exact beat grid, printed bar by bar
- `scripts/track_profile.py <file>...`: loudness range, swell and ending of candidate tracks

FILE: .claude/skills/finishing/scripts/beat_grid.py

4,443 bytes

```
#!/usr/bin/env python3
"""Fit an exact beat grid to a music track and print it bar by bar, to plan a music edit.

HyperFrames' `beats` gives irregular onsets; a cut-on-the-beat edit needs a straight grid.
This fits the period and phase (near --bpm, the track's catalogue tempo) to the track's onset
strength, then prints every bar: its downbeat time, and per beat the level (dB) and the low
band's onset (kick / bass hits). Read it for the song's structure: where the groove enters,
the breaks, and the ending's stop-time hits and final chord.

The downbeat is guessed as the beat phase with the most low-band onset; check it by ear-proxy
(a groove entry or the final chord should land on beat 1) and pass --downbeat to override.
Usage: beat_grid.py <track> --bpm 138 [--beats 4] [--downbeat T] [--json out.json] [--from S --to S]
"""
import argparse
import array
import json
import math
import subprocess

SR, HOP = 8000, 80  # 10 ms frames


def energy(path, af):
    raw = subprocess.run(["ffmpeg", "-v", "error", "-i", path, "-ac", "1", "-ar", str(SR), "-af", af,
                          "-f", "s16le", "-"], capture_output=True, check=True).stdout
    x = array.array("h")
    x.frombytes(raw)
    return [math.sqrt(sum(v * v for v in x[i:i + HOP]) / HOP) / 32768 + 1e-9
            for i in range(0, len(x) - HOP, HOP)]


def onset(e):
    db = [20 * math.log10(v) for v in e]
    return [0.0] + [max(db[i] - db[i - 1], 0.0) for i in range(1, len(db))]


def score(on, period, phase):
    s, t, n = 0.0, phase, 0
    while t < len(on) * HOP / SR:
        i = int(round(t * SR / HOP))
        s += max(on[max(i - 1, 0):i + 2])
        t += period
        n += 1
    return s / max(n, 1)


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("track")
    ap.add_argument("--bpm", type=float, required=True)
    ap.add_argument("--beats", type=int, default=4, help="beats per bar")
    ap.add_argument("--downbeat", type=float, help="a known downbeat time (s)")
    ap.add_argument("--json")
    ap.add_argument("--from", dest="t0", type=float, default=0)
    ap.add_argument("--to", dest="t1", type=float, default=1e9)
    a = ap.parse_args()

    full = energy(a.track, "highpass=f=40")
    low = energy(a.track, "lowpass=f=150")
    on_full, on_low = onset(full), onset(low)
    on = [f + 2 * l for f, l in zip(on_full, on_low)]

    best = (0, None, None)
    for k in range(-60, 61):  # +-1.5 % around the catalogue tempo
        period = 60 / (a.bpm * (1 + k * 0.00025))
        for p in range(int(period * 100)):
            s = score(on, period, p / 100)
            if s > best[0]:
                best = (s, period, p / 100)
    _, period, phase = best
    # Downbeat: the beat phase (of --beats) with the most low-band onset, unless given.
    if a.downbeat is not None:
        first = a.downbeat - math.floor((a.downbeat - phase) / period / a.beats) * a.beats * period
        first = first % (period * a.beats)
    else:
        sums = []
        for b in range(a.beats):
            t, s = phase + b * period, 0.0
            while t < len(on_low) * HOP / SR:
                i = int(round(t * SR / HOP))
                s += max(on_low[max(i - 1, 0):i + 2])
```

```
                t += period * a.beats
            sums.append(s)
        first = phase + sums.index(max(sums)) * period
    print(f"tempo {60 / period:.3f} bpm (beat {period:.4f} s), first downbeat {first:.3f} s")

    grid, bar, t = [], 1, first
    end = len(full) * HOP / SR
    while t < end:
        cells = []
        for b in range(a.beats):
            tb = t + b * period
            i = int(round(tb * SR / HOP))
            if i >= len(full):
                break
            w = full[i:i + int(period * SR / HOP)] or [1e-9]
            lvl = 20 * math.log10(max(w))
            kick = max(on_low[max(i - 1, 0):i + 2])
            cells.append((tb, lvl, kick))
            grid.append({"t": round(tb, 4), "bar": bar, "beat": b + 1, "db": round(lvl, 1),
                         "kick": round(kick, 1)})
        if a.t0 <= t <= a.t1:
            print(f"bar {bar:3d} {t:7.3f}  " + "  ".join(
                f"{c[1]:5.1f}dB{'K' if c[2] > 6 else ('k' if c[2] > 3 else ' ')}" for c in cells))
        t += period * a.beats
        bar += 1
    if a.json:
        json.dump({"bpm": 60 / period, "beat_s": period, "first_downbeat": first, "beats": grid},
                  open(a.json, "w"), indent=1)


if __name__ == "__main__":
    main()
```

FILE: .claude/skills/finishing/scripts/captions.py

17,950 bytes

```
#!/usr/bin/env python3
"""Burn captions from outputs/transcript.cut.json into the newest un-captioned render.

Picks its own input (graphics.mp4, else base.mp4). Never captions a captioned file.
Exits cleanly without writing anything when the style has captions disabled for the format.
"""
import argparse
import json
import re
import subprocess
import sys
from pathlib import Path

INPUT_ORDER = ["graphics.mp4", "base.mp4"]


def root_of(job: Path) -> Path:
    for d in [job.resolve(), *job.resolve().parents]:
        if (d / "brand.md").exists():
            return d
    sys.exit("brand.md not found above the job folder")


def brand_json(root: Path) -> dict:
    m = re.search(r"```json\s*(\{.*?\})\s*```", (root / "brand.md").read_text(), re.S)
    return json.loads(m.group(1)) if m else {}


def resolve(value, brand):
    """$accent -> brand hex, $caption -> brand font, anything else unchanged."""
    if isinstance(value, str) and value.startswith("$"):
        key = value[1:]
        v = brand.get("colors", {}).get(key) or brand.get("fonts", {}).get(key)
        if not v or v == "TODO":
            sys.exit(f"brand.md has no value for {value}. Finish the brand interview first.")
        return v
    return value


def ass_color(hex_color: str) -> str:
    h = hex_color.lstrip("#")
    r, g, b = h[0:2], h[2:4], h[4:6]
    return f"&H00{b}{g}{r}".upper()


def ass_alpha_color(hex_color: str, opacity: float) -> str:
    """ASS colour with alpha: 00 is opaque, FF is transparent."""
```

```
    a = round(255 * (1 - max(0.0, min(1.0, opacity))))
    return f"&H{a:02X}" + ass_color(hex_color)[4:]


def ts(t: float) -> str:
    cs = int(round(t * 100))
    return f"{cs // 360000}:{cs // 6000 % 60:02d}:{cs // 100 % 60:02d}.{cs % 100:02d}"


# Words a caption line should not end on (they lean on the next word), and words a line is
# happy to start with (the natural places to break a phrase).
DANGLING = {"a", "an", "the", "of", "in", "on", "to", "my", "is", "your", "at", "for", "with", "and",
            "no", "are", "such", "very", "was", "were", "be", "that", "just", "they"}
OPENERS = {"and", "with", "where", "so", "in", "but", "because", "that", "which", "like", "or"}


def group(words, lo, hi, max_gap=0.4, min_dur=0.4, fits=lambda text: True):
    """Split words into caption lines of lo-hi words.

    Hard breaks: the end of a sentence, a pause over max_gap, or hidden words in between. Inside each run the
    break points are chosen together (dynamic programming), so the lines read as phrases:
    breaking at sentence punctuation or the end of an audio piece is free, breaking before
    an opener ("and", "with", ...) is cheap, ending a line on a dangling word ("the", "of",
    ...) is expensive, and so is a line on screen for under min_dur (a flash).
    """
    runs, cur = [], []
    for w in words:
        if cur and (w.get("_brk") or w["start"] - cur[-1]["end"] > max_gap
                    or re.search(r"[.!?]$", cur[-1]["word"])):
            runs.append(cur)
            cur = []
        cur.append(w)
    if cur:
        runs.append(cur)

    def bare(w):
        return re.sub(r"[^a-z']", "", w["word"].lower())

    def cost(run, i, j):  # line = run[i:j]
        n = j - i
        c = 0.0 if lo <= n <= hi else (0.5 if n < lo and len(run) < lo else 50.0)
        if run[j - 1]["end"] - run[i]["start"] < min_dur:
            c += 3.0
        if j < len(run):
            last, nxt = run[j - 1], run[j]
            if re.search(r"[.!?,;:]$", last["word"]):
                pass
            elif nxt.get("segment") != last.get("segment"):
                c += 0.3
            elif bare(nxt) in OPENERS:
                c += 1.0
            else:
                c += 2.0
            if bare(last) in DANGLING:
                c += 4.0
            elif bare(last) in OPENERS:
                c += 2.0
        if not fits(" ".join(w.get("_shown", w["word"]) for w in run[i:j])):
            c += 100.0
        return c

    lines = []
    for run in runs:
        best = [0.0] + [float("inf")] * len(run)
        back = [0] * (len(run) + 1)
        for j in range(1, len(run) + 1):
            for i in range(max(0, j - hi), j):
                c = best[i] + cost(run, i, j)
                if c < best[j]:
                    best[j], back[j] = c, i
        cuts, j = [], len(run)
        while j > 0:
            cuts.append((back[j], j))
            j = back[j]
        lines += [run[i:j] for i, j in reversed(cuts)]
    return lines


def silent_segments(job: Path) -> set:
    """Cut-sheet segments whose text is bracketed ("[SOUND: ...]") are sound, not speech."""
    sheet = job / "transcript" / "cutsheet.json"
    if not sheet.exists():
        return set()
    segs = json.loads(sheet.read_text())["segments"]
```

```
    return {n for n, s in enumerate(segs) if s.get("text", "").lstrip().startswith("[")}


def title_windows(job: Path) -> list:
    """Graphics that already show the spoken words (hide_captions) suppress captions."""
    plan = job / "transcript" / "graphics-plan.json"
    if not plan.exists():
        return []
    return [(float(b["start"]), float(b["end"])) for b in json.loads(plan.read_text())["beats"]
            if b.get("hide_captions")]


def hidden(word, silent, windows) -> bool:
    if word.get("segment") in silent:
        return True
    mid = (word["start"] + word["end"]) / 2
    return any(s <= mid < e for s, e in windows)


def graph_args(graph: str, path: Path):
    """ffmpeg args that read the filtergraph from a file, never inline on the command line.

    macOS XProtect blocks ffmpeg launched with a long multi-line filtergraph argument as
    "Malicious Script Blocked" (and kills the terminal with it). A graph file avoids that.
    """
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(graph)
    ver = subprocess.run(["ffmpeg", "-hide_banner", "-version"],
                         capture_output=True, text=True).stdout
    m = re.search(r"ffmpeg version n?(\d+)\.", ver)
    if m and int(m.group(1)) < 7:
        return ["-filter_complex_script", str(path)]
    return ["-/filter_complex", str(path)]


def has_libass() -> bool:
    out = subprocess.run(["ffmpeg", "-hide_banner", "-filters"], capture_output=True, text=True).stdout
    return re.search(r"\ssubtitles\s", out) is not None


def esc(path: Path) -> str:
    """Escape a path for a quoted filter option (spaces are fine inside quotes; ' and : are not)."""
    return str(path).replace("\\", "/").replace(":", "\\:").replace("'", "\\'")


def burn_with_pil(job, src, lines, cfg, brand, font_file, size, color, active, enc):
    """No-libass fallback: one transparent PNG per caption line, drawn with the real font
    (baseline at cfg y, centred, offset drop shadow), overlaid in its window. Same look as
    the ASS path for highlight none/keywords."""
    from PIL import Image, ImageDraw, ImageFont
    w, h = size
    fonts = {}
    sx = sy = int(cfg.get("shadow_px", 0))
    sh = resolve(cfg.get("shadow_color", "#000000"), brand).lstrip("#")
    shadow = (int(sh[0:2], 16), int(sh[2:4], 16), int(sh[4:6], 16),
              round(255 * float(cfg.get("shadow_opacity", 1.0))))
    rgb = lambda c: tuple(int(c.lstrip("#")[i:i + 2], 16) for i in (0, 2, 4)) + (255,)
    y = int(cfg.get("y", int(h * 0.62)))
    tmp = job / "transcript" / "caption-png"
    tmp.mkdir(exist_ok=True)
    for old in tmp.glob("*.png"):
        old.unlink()
    inputs, chain, prev = [], [], "0:v"
    for n, (start, end, words, fs) in enumerate(lines):
        font = fonts.setdefault(fs, ImageFont.truetype(str(font_file), fs))
        img = Image.new("RGBA", (w, h), (0, 0, 0, 0))
        d = ImageDraw.Draw(img)
        text = " ".join(x for x, _ in words)
        x = (w - d.textlength(text, font=font)) / 2
        if sx:
            d.text((x + sx, y + sy), text, font=font, fill=shadow, anchor="ls")
        for word, emph in words:
            d.text((x, y), word, font=font, fill=rgb(active if emph else color), anchor="ls")
            x += d.textlength(word + " ", font=font)
        png = tmp / f"line{n:03d}.png"
        img.save(png)
        inputs += ["-loop", "1", "-t", f"{end + 0.1:.3f}", "-i", str(png)]
        out = "vout" if n == len(lines) - 1 else f"c{n}"
        chain.append(f"[{prev}][{n + 1}:v]overlay=enable='gte(t,{start:.3f})*lt(t,{end:.3f})'"
                     f":eof_action=pass[{out}]")
        prev = out
    subprocess.run(["ffmpeg", "-y", "-hide_banner", "-loglevel", "error", "-i", str(src), *inputs,
                    *graph_args(";\n".join(chain), job / "outputs" / "captioned.filtergraph.txt"),
                    "-map", "[vout]", "-map", "0:a", *enc],
```

```
                  check=True)


def probe_size(path: Path):
    out = subprocess.run(["ffprobe", "-v", "error", "-select_streams", "v:0", "-show_entries",
                          "stream=width,height", "-of", "csv=p=0", str(path)],
                         capture_output=True, text=True, check=True).stdout.strip()
    w, h = out.split(",")
    return int(w), int(h)


def is_captioned(path: Path) -> bool:
    if "caption" in path.name.lower():
        return True
    tags = subprocess.run(["ffprobe", "-v", "error", "-show_entries", "format_tags=comment",
                           "-of", "default=nw=1:nk=1", str(path)],
                          capture_output=True, text=True).stdout
    return "captioned" in tags


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("job", type=Path)
    ap.add_argument("--format", required=True, choices=["long", "short-explainer", "short-raw"])
    ap.add_argument("--style", default="editorial")
    args = ap.parse_args()

    root = root_of(args.job)
    brand = brand_json(root)
    style = json.loads((root / "styles" / args.style / "style.json").read_text())
    cfg = dict(style["captions"].get(args.format, {"enabled": False}))
    # A recipe's own caption rules (the cut sheet names its recipe), e.g. a short's
    # {"break_each_segment": true}: every fragment on its own line.
    sheet = args.job / "transcript" / "cutsheet.json"
    recipe = json.loads(sheet.read_text()).get("recipe") if sheet.exists() else None
    cfg.update(style.get("recipes", {}).get(recipe, {}).get("captions", {}))
    # Per-job overrides (a one-off ask that is not yet a style rule), e.g. {"strip_punctuation": true}
    override = args.job / "transcript" / "captions-override.json"
    if override.exists():
        cfg.update(json.loads(override.read_text()))
    if not cfg.get("enabled"):
        print(f"Captions disabled for {args.format} in style '{args.style}'. Nothing written.")
        return

    outputs = args.job / "outputs"
    src = next((outputs / n for n in INPUT_ORDER if (outputs / n).exists()), None)
    if not src:
        sys.exit("No graphics.mp4 or base.mp4 in outputs/")
    if is_captioned(src):
        sys.exit(f"{src} is already captioned. Refusing to caption it twice.")

    words = json.loads((outputs / "transcript.cut.json").read_text())["words"]
    lo, hi = cfg.get("words_per_line", [2, 4])
    font = resolve(brand.get("fonts", {}).get("caption", "$caption"), brand)
    base_c = ass_color(resolve(cfg["color"], brand))
    act_c = ass_color(resolve(cfg["active_word_color"], brand))
    stroke_c = ass_color(resolve(cfg["stroke_color"], brand))
    # Optional drop shadow: shadow_px offset, shadow_color at shadow_opacity.
    shadow_px = cfg.get("shadow_px", 0)
    shadow_c = ass_alpha_color(resolve(cfg.get("shadow_color", "#000000"), brand),
                               cfg.get("shadow_opacity", 1.0))
    w, h = probe_size(src)

    ass = [
        "[Script Info]", "ScriptType: v4.00+", f"PlayResX: {w}", f"PlayResY: {h}", "",
        "[V4+ Styles]",
        "Format: Name, Fontname, Fontsize, PrimaryColour, OutlineColour, BackColour, Bold, "
        "BorderStyle, Outline, Shadow, Alignment, MarginL, MarginR, MarginV",
        # Bold 0: the brand fonts are single-weight; Bold 1 makes libass fake a heavier cut.
        f"Style: Cap,{font},{cfg.get('size_px', 64)},{base_c},{stroke_c},{shadow_c},0,1,"
        f"{cfg.get('stroke_px', 6)},{shadow_px},2,80,80,{h - int(cfg.get('y', int(h * 0.62)))}",
        "", "[Events]", "Format: Layer, Start, End, Style, Text",
    ]
    # highlight: "karaoke" colours each word as it is spoken; "keywords" colours only the
    # word indices listed in transcript/caption-emphasis.json ({"indices": [...]}), for the
    # whole time their line is on screen; "none" colours nothing.
    mode = cfg.get("highlight", "karaoke")
    emphasis = set()
    if mode == "keywords":
        ep = args.job / "transcript" / "caption-emphasis.json"
        emphasis = set(json.loads(ep.read_text()).get("indices", [])) if ep.exists() else set()
    silent, windows = silent_segments(args.job), title_windows(args.job)
    segs = json.loads(sheet.read_text())["segments"] if sheet.exists() else []
    kept, skipped = [], False
```

```
for wd in words:
    if hidden(wd, silent, windows):
        skipped = True
        continue
    wd["_brk"], skipped = skipped, False  # a caption never bridges hidden words
    if cfg.get("break_each_segment") and kept and kept[-1].get("segment") != wd.get("segment"):
        # One line per cut-sheet piece (a fragment said in one go), unless the piece is
        # marked "caption_joins_next": one phrase the pause trim split into two pieces.
        prev = kept[-1].get("segment")
        if not (isinstance(prev, int) and prev < len(segs) and segs[prev].get("caption_joins_next")):
            wd["_brk"] = True
    kept.append(wd)
words = kept
# {"break_before": ["new york", ...]}: start a new line on each occurrence of a phrase
# (lowercase, punctuation ignored), for phrase boundaries the chunker cannot see, such as a
# line that runs across a section cut, or keeping "35 to 45 dollars" together.
bare_words = [re.sub(r"[^a-z0-9']", "", w["word"].lower()) for w in words]
for phrase in cfg.get("break_before", []):
    p = phrase.lower().split()
    for n in range(len(words) - len(p) + 1):
        if bare_words[n:n + len(p)] == p:
            words[n]["_brk"] = True
for n, word in enumerate(words):
    word["_i"] = n
    if cfg.get("uppercase"):
        word["word"] = word["word"].upper()
    word["_shown"] = re.sub(r"[.,!?;:\"]", "", word["word"]) if cfg.get("strip_punctuation") \
        else word["word"]

def paint(x, on):
    return f"{{\\c{act_c}}}{x['_shown']}{{\\c{base_c}}}" if on else x["_shown"]

plain = []  # (start, end, [(word, emphasised)]) for the non-libass path
fits = lambda text: True
line_size = lambda text: int(cfg.get("size_px", 64))
try:
    from PIL import ImageFont
    size_px = int(cfg.get("size_px", 64))
    min_px = int(cfg.get("min_size_px", size_px))  # shrink a long phrase to fit, down to this
    font_path = str(root / brand["font_files"]["caption"][0])
    cap_font = ImageFont.truetype(font_path, min_px)
    fits = lambda text: cap_font.getlength(text.upper() if cfg.get("uppercase") else text) <= w - 160

    def line_size(text):
        s = size_px
        while s > min_px and ImageFont.truetype(font_path, s).getlength(text) > w - 160:
            s -= 2
        return s
except (ImportError, KeyError, OSError):
    pass
vid_dur = float(subprocess.run(["ffprobe", "-v", "error", "-show_entries", "format=duration", "-of", "csv=p=0",
                                str(src)], capture_output=True, text=True).stdout.strip() or 0)
for line in group(words, lo, hi, fits=fits):
    # A line ending a frame or two before a hard cut out (a loop point) holds to the last
    # frame; otherwise the final frame blinks to no caption.
    if vid_dur and 0 < vid_dur - line[-1]["end"] < 0.12:
        line[-1] = dict(line[-1], end=vid_dur)
    end = min([line[-1]["end"], *[s for s, _ in windows if s >= line[0]["start"]]])
    shown = " ".join(x["_shown"] for x in line)
    fs = line_size(shown)
    fs_tag = f"{{\\fs{fs}}}" if fs != int(cfg.get("size_px", 64)) else ""
    plain.append((line[0]["start"], end, [(x["_shown"], x["_i"] in emphasis) for x in line], fs))
    if mode == "karaoke":
        for i, active in enumerate(line):
            end = line[i + 1]["start"] if i + 1 < len(line) else active["end"]
            text = " ".join(paint(x, j == i) for j, x in enumerate(line))
            ass.append(f"Dialogue: 0,{ts(active['start'])},{ts(end)},Cap,{fs_tag}{text}")
    else:
        text = " ".join(paint(x, x["_i"] in emphasis) for x in line)
        ass.append(f"Dialogue: 0,{ts(line[0]['start'])},{ts(line[-1]['end'])},Cap,{fs_tag}{text}")

ass_path = args.job / "transcript" / "captions.ass"
ass_path.write_text("\n".join(ass) + "\n")
out = outputs / "captioned.mp4"
enc = ["-c:v", "libx264", "-crf", "18", "-preset", "medium", "-pix_fmt", "yuv420p",
       "-c:a", "copy", "-metadata", "comment=captioned", str(out)]
if has_libass():
    fonts_dir = root / "assets" / "fonts"
    vf = f"subtitles=filename='{esc(ass_path.resolve())}':fontsdir='{esc(fonts_dir.resolve())}'"
    subprocess.run(["ffmpeg", "-y", "-hide_banner", "-loglevel", "error", "-i", str(src),
                    "-vf", vf, *enc], check=True)
else:
    if mode == "karaoke":
        sys.exit("This ffmpeg has no libass (subtitles filter) and karaoke needs it: "
```

```
                    "install an ffmpeg built with libass.")
        font_file = root / brand["font_files"]["caption"][0]
        burn_with_pil(args.job, src, plain, cfg, brand, font_file, (w, h),
                      resolve(cfg["color"], brand), resolve(cfg["active_word_color"], brand), enc)
    print(f"{src.name} -> {out} ({len(words)} words)")


if __name__ == "__main__":
    main()
```

FILE: .claude/skills/finishing/scripts/find_release.py

2,272 bytes

```
#!/usr/bin/env python3
"""Find candidate "release" moments in a music track: where the music opens up after a build.

Measures short-term loudness (EBU R128, 3 s window) every 0.1 s and ranks the moments with
the biggest sustained rise: the mean over the next 4 s minus the mean over the previous 4 s.
Prints the top candidates as track times. Always listen/verify: a release can also be a
change in texture or harmony that loudness does not show.

Usage: find_release.py <track> [--top 5] [--min-gap 6]
"""
import argparse
import re
import subprocess


def loudness_curve(path):
    err = subprocess.run(
        ["ffmpeg", "-hide_banner", "-nostats", "-v", "verbose", "-i", path, "-af", "ebur128",
         "-f", "null", "-"], capture_output=True, text=True).stderr
    pts = []
    for m in re.finditer(r"t:\s*([\d.]+)\s+TARGET.*?S:\s*(-?[\d.]+|-inf)", err):
        s = m.group(2)
        pts.append((float(m.group(1)), -70.0 if s == "-inf" else max(float(s), -70.0)))
    return pts


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("track")
    ap.add_argument("--top", type=int, default=5)
    ap.add_argument("--min-gap", type=float, default=6.0, help="seconds between candidates")
    args = ap.parse_args()

    pts = loudness_curve(args.track)
    if len(pts) < 100:
        raise SystemExit("track too short to analyse")
    step = pts[1][0] - pts[0][0]
    win = max(int(round(4.0 / step)), 1)
    scored = []
    # Skip the first 3 s: the short-term window is still filling from silence there.
    first = next((i for i, (t, _) in enumerate(pts) if t >= 3.0), 0) + win
    for i in range(first, len(pts) - win):
        before = sum(v for _, v in pts[i - win:i]) / win
        after = sum(v for _, v in pts[i:i + win]) / win
        scored.append((after - before, pts[i][0], before, after))
    scored.sort(reverse=True)

    picked = []
    for rise, t, before, after in scored:
        if rise <= 0 or len(picked) >= args.top:
            break
        if all(abs(t - p[1]) >= args.min_gap for p in picked):
            picked.append((rise, t, before, after))
    for rise, t, before, after in picked:
        print(f"track_t ~{t:6.1f}s  rise {rise:+5.1f} LU  ({before:.1f} -> {after:.1f} LUFS)")


if __name__ == "__main__":
    main()
```

FILE: .claude/skills/finishing/scripts/mix.py

12,182 bytes

```
#!/usr/bin/env python3
"""Apply transcript/audio-plan.json: music bed pass, then SFX pass.

Pure audio passes: the video stream is copied, never re-encoded.
Each pass picks its own input (the newest prior-stage file) and writes the exact
```

```
filtergraph it used to transcript/audio-filtergraph.txt so it can be re-applied verbatim.
"""
import argparse
import json
import re
import subprocess
import sys
from pathlib import Path

MUSIC_INPUTS = ["captioned.mp4", "graphics.mp4", "base.mp4"]
# Every pass that sums tracks ends on a limiter: a bed or a hit on top of a speech track that
# already peaks at -1.5 dBTP pushes it over. -1.9 dBFS sample peak keeps true peak under -1.
# level=disabled: alimiter otherwise normalises back up to 0 dB.
LIMIT = "alimiter=limit=0.8:level=disabled"
SFX_INPUTS = ["music.mp4", *MUSIC_INPUTS]


def root_of(job: Path) -> Path:
    for d in [job.resolve(), *job.resolve().parents]:
        if (d / "brand.md").exists():
            return d
    sys.exit("brand.md not found above the job folder")


def duration(path: Path) -> float:
    """The picture's length (frames / rate): the container adds AAC padding past the last
    frame, and audio running past the picture leaves a stray tail at a Short's loop point."""
    out = subprocess.run(["ffprobe", "-v", "error", "-select_streams", "v:0", "-count_packets",
                          "-show_entries", "stream=nb_read_packets,r_frame_rate", "-of", "csv=p=0",
                          str(path)], capture_output=True, text=True, check=True).stdout.strip()
    rate, n = out.split(",")[:2]
    num, den = rate.split("/")
    return int(n) * int(den) / int(num)


def pick(outputs: Path, order):
    for n in order:
        if (outputs / n).exists():
            return outputs / n
    sys.exit(f"none of {order} exist in {outputs}")


def resolve_file(job: Path, root: Path, rel: str) -> Path:
    for base in (job, root):
        if (base / rel).exists():
            return base / rel
    sys.exit(f"audio file not found: {rel}")


def gain(db):
    return 10 ** (float(db) / 20)


def pieces_chain(m, dur, inp=1, tag=""):
    """A music edit: stretches of the track laid at set video times.

    Each piece plays track_t onward from video_t until the next piece starts (the last one
    until the end). Joins crossfade over xfade_ms, centred on the join. A piece may carry
    db_offset, relative to the bed level. Used to stitch a
    track's own ending onto the body so the video ends on the music's final hit.
    """
    ps = sorted(m["pieces"], key=lambda p: float(p["video_t"]))
    x = float(m.get("xfade_ms", 20)) / 1000
    parts, labels = [], []
    for i, p in enumerate(ps):
        vt, tt = float(p["video_t"]), float(p["track_t"])
        end_v = float(ps[i + 1]["video_t"]) if i + 1 < len(ps) else dur
        a = max(vt - (x / 2 if i else 0), 0)          # overlap the neighbour by half the fade
        b = min(end_v + (x / 2 if i + 1 < len(ps) else 0), dur)
        if m.get("_track_len"):
            # The music input is looped (-stream_loop); a piece must never run past the end
            # of the file, or the track restarts audibly. A track cut to end early just ends.
            b = min(b, a + float(m["_track_len"]) - (tt - (vt - a)))
        chain = (f"[{inp}:a]atrim=start={tt - (vt - a):.4f}:duration={b - a:.4f},asetpts=PTS-STARTPTS")
        if i:
            chain += f",afade=t=in:d={x:.4f}"
        if i + 1 < len(ps):
            chain += f",afade=t=out:st={b - a - x:.4f}:d={x:.4f}"
        if float(p.get("db_offset", 0)):
            chain += f",volume={float(p['db_offset'])}dB"  # e.g. keep a louder ending under the voice
        chain += f",adelay={round(a * 1000)}:all=1[{tag}p{i}]"
        parts.append(chain)
        labels.append(f"[{tag}p{i}]")
    # asetpts: once one input ends, amix emits frames with no timestamps, and anything
```

```
    # downstream that reads t (the envelope) silently sees NaN. Rebuild them from the samples.
    parts.append("".join(labels) + f"amix=inputs={len(ps)}:duration=longest:normalize=0,"
                 f"asetpts=N/SR/TB,atrim=0:{dur:.3f}[{tag}mus]")
    return ";".join(parts) + f";[{tag}mus]"


def envelope(points):
    """Piecewise-linear gain in dB over video time, e.g. [[1.0, 4], [8.5, 4], [10.7, 0]]:
    lift a quiet intro so it is audible, easing back before the groove."""
    pts = sorted((float(t), float(d)) for t, d in points)
    expr = f"{pts[-1][1]}"
    for (t0, d0), (t1, d1) in reversed(list(zip(pts, pts[1:]))):
        seg = f"({d0}+({d1}-({d0}))*(t-{t0})/{t1 - t0:.4f})"
        expr = f"if(lt(t,{t1}),{seg},{expr})"
    expr = f"if(lt(t,{pts[0][0]}),{pts[0][1]},{expr})"
    return f"volume='pow(10,({expr})/20)':eval=frame"


def music_graph(m, dur):
    if isinstance(m, list):
        # Several beds (a long video: each section's track, the bridges' tracks), summed. Each bed is a
        # pieces edit on its own input with its own envelope; set fade_out 0 on beds that end early.
        chains = [bed_chain(b, dur, inp=k + 1, tag=f"b{k}") for k, b in enumerate(m)]
        labels = "".join(f"[b{k}bed]" for k in range(len(m)))
        return ";".join(chains) + f";[0:a]{labels}amix=inputs={len(m) + 1}:duration=first:normalize=0,{LIMIT}[aout]
» "
    chain = bed_chain(m, dur)
    if m.get("duck"):
        return chain + ";[bed][0:a]sidechaincompress=threshold=0.05:ratio=6[bd];" \
                       f"[0:a][bd]amix=inputs=2:duration=first:normalize=0,{LIMIT}[aout]"
    return chain + f";[0:a][bed]amix=inputs=2:duration=first:normalize=0,{LIMIT}[aout]"


def bed_chain(m, dur, inp=1, tag=""):
    db = m.get("db", -18)
    rel = m.get("release")
    if m.get("pieces"):
        chain = pieces_chain(m, dur, inp, tag) + f"volume={db}dB"
        if m.get("envelope"):
            chain += "," + envelope(m["envelope"])
    elif not rel:
        chain = f"[{inp}:a]atrim=0:{dur:.3f},volume={db}dB"
    else:
        # Line the track's release moment (track_t) up with the video's (video_t): trim the
        # track's head, or delay the track's start. Before the release the bed sits at pre_db,
        # then eases to db over ramp_s. The "build then release" is in the music itself;
        # the level change only settles the bed under the talk once the argument starts.
        vt, tt = float(rel["video_t"]), float(rel["track_t"])
        pre, ramp = gain(rel.get("pre_db", db)), float(rel.get("ramp_s", 1.0))
        post = gain(db)
        chain = f"[{inp}:a]"
        if tt >= vt:
            chain += f"atrim=start={tt - vt:.3f},asetpts=PTS-STARTPTS,"
        else:
            chain += f"adelay={int(round((vt - tt) * 1000))}:all=1,"
        expr = (f"if(lt(t,{vt:.3f}),{pre:.5f},if(lt(t,{vt + ramp:.3f}),"
                f"{pre:.5f}+({post:.5f}-{pre:.5f})*(t-{vt:.3f})/{ramp:.3f},{post:.5f}))")
        chain += f"atrim=0:{dur:.3f},volume='{expr}':eval=frame"
    if m.get("fade_in"):
        chain += ",afade=t=in:d=1.0"
    fo = float(m.get("fade_out", 1.5))
    if fo > 0:  # 0 = cut, not faded (a looping Short must not sound like it ends)
        chain += f",afade=t=out:st={max(dur - fo, 0):.3f}:d={fo:.3f}"
    return chain + f"[{tag}bed]"


def graph_args(graph: str, path: Path):
    """ffmpeg args that read the filtergraph from a file, never inline on the command line.

    macOS XProtect blocks ffmpeg launched with a long multi-line filtergraph argument as
    "Malicious Script Blocked" (and kills the terminal with it). A graph file avoids that.
    """
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(graph)
    ver = subprocess.run(["ffmpeg", "-hide_banner", "-version"],
                         capture_output=True, text=True).stdout
    m = re.search(r"ffmpeg version n?(\d+)\.", ver)
    if m and int(m.group(1)) < 7:
        return ["-filter_complex_script", str(path)]
    return ["-/filter_complex", str(path)]


def sfx_graph(sfx):
    parts, labels = [], ["[0:a]"]
```

```
    for i, s in enumerate(sfx, start=1):
        ms = int(round(float(s["t"]) * 1000))
        # Optional "in"/"out" (seconds in the file): lay only that stretch, starting at t.
        trim = ""
        if "in" in s or "out" in s:
            trim = f"atrim=start={float(s.get('in', 0))}" + (f":end={float(s['out'])}" if "out" in s else "") + ",a
» setpts=PTS-STARTPTS,"
        # Optional "fade_out" (s): dissolve the tail of the laid stretch (needs "out") instead of a hard stop.
        if s.get("fade_out") and "out" in s:
            fo, d = float(s["fade_out"]), float(s["out"]) - float(s.get("in", 0))
            trim += f"afade=t=out:st={max(d - fo, 0):.3f}:d={fo:.3f},"
        parts.append(f"[{i}:a]{trim}aformat=sample_rates=48000:channel_layouts=stereo,volume={s.get('db', -10)}dB,a
» delay={ms}:all=1[s{i}]")
        labels.append(f"[s{i}]")
    parts.append("".join(labels) + f"amix=inputs={len(labels)}:duration=first:normalize=0,{LIMIT}[aout]")
    return ";".join(parts)


def run(src, extra_inputs, graph, out, log, label, dry, loop=()):
    cmd = ["ffmpeg", "-y", "-hide_banner", "-loglevel", "error", "-i", str(src)]
    for f in extra_inputs:
        cmd += [*loop, "-i", str(f)]
    cut = graph.rfind("[aout]")  # end the audio with the picture, to the frame
    graph = graph[:cut] + f",atrim=0:{duration(src):.4f}" + graph[cut:]
    # -map_chapters -1: a licensed track's chapter tags would otherwise come along as a stray
    # text stream as long as the song.
    cmd += [*graph_args(graph, out.with_suffix(".filtergraph.txt")), "-map", "0:v", "-map", "[aout]",
            "-map_chapters", "-1", "-c:v", "copy", "-c:a", "aac", "-b:a", "256k", str(out)]
    log.append(f"# {label}: {src.name} -> {out.name}\n{graph}\n")
    print(f"{label}: {src.name} -> {out.name}")
    if not dry:
        subprocess.run(cmd, check=True)


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("job", type=Path)
    g = ap.add_mutually_exclusive_group()
    g.add_argument("--music-only", action="store_true")
    g.add_argument("--sfx-only", action="store_true")
    ap.add_argument("--dry-run", action="store_true")
    ap.add_argument("--plan", type=Path, help="audio plan (default transcript/audio-plan.json)")
    ap.add_argument("--src", type=Path, help="input video (default: newest prior-stage file)")
    ap.add_argument("--prefix", default="", help="output name prefix, e.g. episode- -> outputs/episode-music.mp4")
    args = ap.parse_args()

    root = root_of(args.job)
    plan = json.loads((args.plan or args.job / "transcript" / "audio-plan.json").read_text())
    outputs = args.job / "outputs"
    log = []

    if not args.sfx_only:
        m = plan.get("music")
        beds = m if isinstance(m, list) else [m]
        if not m or not all(b and b.get("file") for b in beds):
            sys.exit("No music file in audio-plan.json. Ask the user for a licensed track; never download one.")
        src = args.src or pick(outputs, MUSIC_INPUTS)
        tracks = []
        for b in beds:
            tracks.append(resolve_file(args.job, root, b["file"]))
            b["_track_len"] = float(subprocess.run(
                ["ffprobe", "-v", "error", "-show_entries", "format=duration", "-of", "default=nw=1:nk=1",
                 str(tracks[-1])], capture_output=True, text=True, check=True).stdout)
        run(src, tracks, music_graph(m, duration(src)),
            outputs / f"{args.prefix}music.mp4", log, "music", args.dry_run, loop=("-stream_loop", "-1"))

    if not args.music_only:
        sfx = sorted(plan.get("sfx") or [], key=lambda s: float(s["t"]))
        if not sfx:
            print("No SFX in plan. Skipping (never synthesise a tone).")
        else:
            src = (outputs / f"{args.prefix}music.mp4" if (not args.sfx_only and not args.dry_run)
                   else args.src or pick(outputs, SFX_INPUTS))
            files = [resolve_file(args.job, root, s["file"]) for s in sfx]
            run(src, files, sfx_graph(sfx), outputs / f"{args.prefix}sfx.mp4", log, "sfx", args.dry_run)

    if log and not args.dry_run:
        (args.job / "transcript" / "audio-filtergraph.txt").write_text("\n".join(log))
    elif log:
        print("\n".join(log))


if __name__ == "__main__":
    main()
```

FILE: .claude/skills/finishing/scripts/track_profile.py

2,048 bytes

```
#!/usr/bin/env python3
"""Profile candidate music tracks without listening: loudness range, how busy and how swelling
they are, and how they end.

For each file prints integrated loudness and LRA (a large LRA means big dynamic swells, which
reads as theatrical under a voice), the spread of 2 s short-term levels, and the ending: the
level of the last 8 s in 0.5 s steps, so a track that finishes on a clean final hit (a sharp
drop to silence) can be told from one that fades or stops mid-phrase.
Usage: track_profile.py <file> [<file> ...]
"""
import re
import subprocess
import sys


def main():
    for f in sys.argv[1:]:
        r = subprocess.run(["ffmpeg", "-hide_banner", "-nostats", "-i", f, "-af", "ebur128", "-f", "null", "-"],
                           capture_output=True, text=True).stderr
        summary = r[r.rfind("Summary:"):]  # the per-frame lines come first
        i = re.search(r"I:\s+(-?[\d.]+) LUFS", summary)
        lra = re.search(r"LRA:\s+([\d.]+) LU", summary)
        st = [float(m) for m in re.findall(r" S:\s*(-?[\d.]+)", r) if float(m) > -70]
        dur = float(subprocess.run(["ffprobe", "-v", "error", "-show_entries", "format=duration", "-of",
                                    "csv=p=0", f], capture_output=True, text=True).stdout)
        tail = []
        t = max(dur - 8, 0)
        while t < dur - 0.25:
            out = subprocess.run(["ffmpeg", "-hide_banner", "-ss", f"{t:.2f}", "-t", "0.5", "-i", f, "-af",
                                  "volumedetect", "-f", "null", "-"], capture_output=True, text=True).stderr
            m = re.search(r"max_volume: (-?[\d.]+) dB", out)
            tail.append(f"{float(m.group(1)):.0f}" if m else "-inf")
            t += 0.5
        spread = (max(st) - min(st)) if st else 0
        print(f"{f.split('/')[-1][:40]:40s} {dur:6.1f}s  I {i.group(1) if i else '?'}  "
              f"LRA {lra.group(1) if lra else '?'}  short-term spread {spread:.0f} dB")
        print(f"   last 8 s (0.5 s max dB): {' '.join(tail)}")


if __name__ == "__main__":
    main()
```

Skill: export

FILE: .claude/skills/export/SKILL.md

2,435 bytes

```
---
name: export
description: Stage 5. Turn a messy outputs folder into one unambiguous final file - promote the newest real render
» to projects/<job>/outputs/<job>.mp4, retire superseded drafts, keep what's needed to reopen the job, copy the final
»  to Downloads - and optionally reclaim scratch space. Dry run by default. Use when the user says the video is done,
»  asks to export, ship, finalise or clean up a job.
---

# Export

Its whole job: turn a messy outputs folder into one unambiguous file.

By the end of a job there is a base cut, a graphics pass, a captions pass, a music pass,
an SFX pass and some drafts, and in a week nobody will know which one shipped.

## Promote (`scripts/export.py promote <job>`)

- Promotes the **newest real render** (by modification time, among `sfx.mp4`, `music.mp4`,
  `captioned.mp4`, `graphics.mp4`, `base.mp4`, and any other `.mp4` in `outputs/`) to
  `outputs/<job>.mp4`, a single clearly named final. The job name is the folder name, so
  the final is named after the content.
- **Retires** the superseded drafts (deletes them).
- **Keeps**, always: `base.mp4` (so graphics can be re-composited), `transcript.cut.json`,
  everything in `transcript/`, and `graphics-build/` source, so the job can be reopened.
- Copies the final to `~/Downloads/` (override with `--copy-to`).

## Reclaim (`scripts/export.py clean <job>`)

Separate and optional. Deletes render scratch: `graphics-build/renders/`, cached intermediate
renders, stray `node_modules/`. **Never** source footage (`raw/`, `broll/`, `audio/`,
`assets/`), **never** anything in `outputs/`.

## Two rules that make it safe to trust

1. **Dry run by default, always.** Both commands print exactly what they would promote,
   delete and keep, and do nothing until `--apply` is passed. Show the plan to the user and
   get a yes before re-running with `--apply`.
2. **Never delete anything newer than the deliverable.** The guard is inside the script,
   comparing modification times, and aborts the whole operation. Do not work around it: if it
   trips, something was rendered after the file you think is final. Ask the user.

## Close out the job

Before calling the job done, go back through the final-review notes. Any correction that
should apply to every future video gets written into `styles/<style>/style.md` and the
`learned` array in `style.json`. Show the user the diff first. One-off notes are just applied
and forgotten.
```

FILE: .claude/skills/export/scripts/export.py

5,093 bytes

```
#!/usr/bin/env python3
"""Promote the newest render to one final file, retire drafts, optionally reclaim scratch.

Dry run by default: prints the plan and changes nothing until --apply.
Guard (not overridable): never delete anything newer than the deliverable.
"""
import argparse
import shutil
import sys
from pathlib import Path

KEEP_IN_OUTPUTS = {"base.mp4", "transcript.cut.json"}
NEVER_TOUCH = {"raw", "broll", "audio", "assets", "outputs", "transcript"}
SCRATCH = ["graphics-build/renders", "graphics-build/.cache", "graphics-build/node_modules"]


def size(p: Path) -> int:
    if p.is_file():
        return p.stat().st_size
    return sum(f.stat().st_size for f in p.rglob("*") if f.is_file())


def human(n: int) -> str:
    for unit in ("B", "KB", "MB", "GB"):
        if n < 1024:
```

```
            return f"{n:.0f}{unit}"
        n /= 1024
    return f"{n:.1f}TB"


def guard(deliverable: Path, victims):
    limit = deliverable.stat().st_mtime
    newer = [v for v in victims if v.stat().st_mtime > limit]
    if newer:
        print(f"ABORT: these are newer than the deliverable {deliverable.name}:")
        for v in newer:
            print(f"  {v}")
        print("Something was rendered after the file being promoted. Nothing was changed.")
        sys.exit(2)


def promote(job: Path, apply: bool, copy_to: Path):
    outputs = job / "outputs"
    final = outputs / f"{job.resolve().name}.mp4"
    # Skip macOS AppleDouble files ("._music.mp4"): on exFAT/FAT drives every file gets one,
    # it is written at the same moment as the real file, and it would win "newest" and be
    # promoted while the real render was deleted as a draft.
    renders = [p for p in outputs.glob("*.mp4") if p != final and not p.name.startswith("._")]
    if not renders:
        if final.exists():
            print(f"Already exported: {final}")
            return
        sys.exit(f"No renders in {outputs}")
    newest = max(renders, key=lambda p: p.stat().st_mtime)
    if final.exists() and final.stat().st_mtime > newest.stat().st_mtime:
        sys.exit(f"{final.name} is newer than every render; nothing to promote.")

    retire = [p for p in renders if p != newest and p.name not in KEEP_IN_OUTPUTS]
    if final.exists():
        retire.append(final)  # superseded earlier final
    keep = sorted(p for p in outputs.iterdir() if p.name in KEEP_IN_OUTPUTS and p != newest)

    guard(newest, retire)

    print(f"PROMOTE  {newest.name} -> {final.name}  ({human(size(newest))})")
    for p in retire:
        print(f"DELETE   {p.relative_to(job)}  ({human(size(p))})")
    for p in keep:
        print(f"KEEP     {p.relative_to(job)}")
    print("KEEP     transcript/, graphics-build/ source, raw/, broll/, audio/, assets/")
    print(f"COPY     {final.name} -> {copy_to}/")

    if not apply:
        print("\nDry run. Nothing changed. Re-run with --apply after reading the plan.")
        return
    if newest.name in KEEP_IN_OUTPUTS:
        shutil.copy2(newest, final)
    else:
        newest.replace(final)
    for p in retire:
        if p != final:
            p.unlink()
    copy_to.mkdir(parents=True, exist_ok=True)
    shutil.copy2(final, copy_to / final.name)
    print(f"\nDone: {final} (copy in {copy_to})")


def clean(job: Path, apply: bool):
    final = job / "outputs" / f"{job.resolve().name}.mp4"
    if not final.exists():
        sys.exit("No exported final yet. Run promote first.")
    victims = []
    for rel in SCRATCH:
        p = job / rel
        if p.exists():
            victims.append(p)
    for nm in job.rglob("node_modules"):
        top = nm.relative_to(job).parts[0]
        if top not in NEVER_TOUCH and nm not in victims and not any(v in nm.parents for v in victims):
            victims.append(nm)
    for v in victims:
        if v.relative_to(job).parts[0] in NEVER_TOUCH:
            sys.exit(f"refusing: {v} is protected")
    files = [f for v in victims for f in ([v] if v.is_file() else v.rglob("*")) if f.is_file()]
    guard(final, files)

    total = 0
    for v in victims:
        total += size(v)
```

```
        print(f"DELETE   {v.relative_to(job)}/  ({human(size(v))})")
    print(f"Reclaims {human(total)}. Never touches raw/, broll/, audio/, assets/, outputs/, transcript/.")
    if not victims:
        print("Nothing to reclaim.")
        return
    if not apply:
        print("\nDry run. Nothing changed. Re-run with --apply after reading the plan.")
        return
    for v in victims:
        shutil.rmtree(v) if v.is_dir() else v.unlink()
    print("Done.")


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("action", choices=["promote", "clean"])
    ap.add_argument("job", type=Path)
    ap.add_argument("--apply", action="store_true")
    ap.add_argument("--copy-to", type=Path, default=Path.home() / "Downloads")
    args = ap.parse_args()
    if not (args.job / "outputs").is_dir():
        sys.exit(f"{args.job}/outputs not found")
    if args.action == "promote":
        promote(args.job, args.apply, args.copy_to)
    else:
        clean(args.job, args.apply)


if __name__ == "__main__":
    main()
```

Skill: premiere-xml-cut

FILE: .claude/skills/premiere-xml-cut/SKILL.md

9,123 bytes

```
---
name: premiere-xml-cut
description: Stage 1 (Premiere route). Read the transcripts of a multi-camera shoot, work out the storyline and sha
» re it with the user, then build a Premiere Pro rough-cut XML (FCP7 xmeml) with a story-first cut, the style's openi
» ng and hook, synced camera tracks, B-roll covers, on-screen callouts, a suggested-B-roll track and every unused cli
» p for cross-reference. Use when the user wants the edit handed to Premiere instead of an FFmpeg render, or asks to
» turn a transcript or footage folder into a Premiere timeline or XML.
---

# Premiere XML cut

Its whole job: turn a shoot's transcripts into a story the user agrees with, then into a
Premiere timeline they can open, relink and refine. Claude cannot open Premiere, so every
decision is made from transcripts, frames and file metadata, and the XML is validated by
script before the user sees it.

Use this instead of `rough-cut` when the user says "Premiere", "XML" or "timeline", or when
the shoot has several cameras that need to stay editable. The transcription step is shared
with `rough-cut` (`scripts/transcribe.sh`, run once per video).

## Inputs and outputs

| Reads                                                | Writes                                                      |
|------------------------------------------------------|-------------------------------------------------------------|
| every camera folder in `projects/<job>/`             | `transcript/transcript.json`, `transcript/cutsheet.json` |
| `brand.md`, `styles/<style>/style.md` + `style.json` | `outputs/<job>-rough-cut.xml`                               |
| `assets/fonts`, the style's callout spec             | `outputs/<job>-hook-montage.xml` (only if asked)            |
| the user's notes and corrections                     | `graphics-build/callouts/*.png`                             |

There are no shared scripts for this skill: each shoot's cut data differs. Write the job's
scripts in `projects/<job>/tmp/` (all paths relative to the project, never absolute):
`build_xml.py` (cut data + XML writer), `validate_xml.py`, `check_sync.py`,
`make_callouts.py`, and `make_sync_test.py` when a new camera appears. Keep the cut data
(order, covers, opener) in one clearly marked block at the top of `build_xml.py`.

## Steps

1. **Read the taste files.** `brand.md`, the style's `style.md` (its opening, callout,
   colour and workflow sections) and `style.json`. Colours as tokens, never hex.
2. **Inventory.** `ffprobe` every clip: camera, fps, resolution, duration, audio channels.
   Check whether the voice camera's audio is dual mono (left and right identical). If it
   is, the timeline gets one voice track, never two.
3. **Transcribe once** with WhisperX, B-roll takes included (creators narrate direction in
   them). Run the mishear pass from `rough-cut` step 3.
4. **Understand the story, then share it. Stop and wait.** Read every transcript and write,
   in plain words:
   - the story in a few lines, and the sections in the proposed order with a one-line
     reason each
   - per section: the key point, the most vivid detail and the payoff
   - which clips are voiceover, which are face to camera, which are B-roll narration
   - the length, and a cut-first list if it runs long
   - what is missing (a promised shot that was never filmed, no sign-off, a jump cut that
     needs cover)

   Ask for: section order, must-keep lines, voiceover pickups, music if relevant. Put every
   question the user must answer on its own line at the end. Do not build until they answer.
5. **Write the cutsheet.** `transcript/cutsheet.json`, one entry per cut with a `text` field.
   Reading the texts top to bottom must read as the video. Never cut mid-sentence: check the
   words past every cut.
6. **Sync the cameras.** Offset by transcript words, refined by GCC-PHAT cross-correlation on
   the audio (`camB_t = camA_t + offset`). Check per-cut word overlap and a median residual
   near zero. Cameras that never share audio (B-roll only) are not synced and are used as
   cover.
7. **Open the way the style says**, then the hook. If the style asks for an opening that
   shows a specific moment, check frames for each shot before using it. If the shoot has
   no real version of the promised shot, say so and offer the closest one.
8. **Cover with B-roll.** Anchor each cover to the spoken phrase using word timings, not to
   a guessed second. Show the thing being named at the moment it is named, and mix with
   action. Keep the speaker's face on the camera they are looking at. Anywhere the voice
   continues under a cover it stays on the voice track; only the picture is carved.
9. **Callouts.** One PNG per named item the style wants called out (a product, a place, a
   person), with placeholder text where a detail is unknown, rendered from the style's
   callout spec and the brand fonts, on their own track at the item's first mention.
10. **Suggested B-roll.** A switched-off track of tag-scored cover candidates that need not
    be audio synced. Say plainly that windows come from tags and stills, not from watching.
11. **Build the XML** with the conventions below, then run the validator. It must report no
    overlaps, range errors, dangling links, missing files, or file references before their
```

    definition.
12. **Test unfamiliar cameras first.** The first time a new camera or frame rate appears,
    send a two-clip test XML in a few in/out-point variants and ask the user which variant
    lines up. Only then build the full file.
13. **Hand over** with the handoff note below, questions last.

XML conventions (each one cost a failed import or a rework)

- **Names:** clip name is the exact source filename, because the user relinks by name.
- **Tracks:** one video track per camera so colour is per camera (V1 voice camera, then one
  track per other camera), then callouts, then switched-off alternates, then switched-off
  suggested B-roll. One audio track when the voice is dual mono.
- **`sourcetrack` trackindex is 1 for every video clip.** 2 or 3 makes the import fail with an
  empty error dialog.
- **No overlaps on a track.** Trim after all items exist, with an explicit final pass.
- **In and out points:** by spec they are frames in the clip's own rate, but Premiere has
  read 23.976 fps clips at the sequence rate (25 per source second) in a 25 fps sequence.
  Confirm with the test in step 12 for any new frame-rate mix. Compute clip and file
  durations with one function.
- **File definitions** appear once, in full; later uses are `<file id=.../>` references and
  must come after the definition.
- **Disabled clips** use `<enabled>FALSE</enabled>`.
- **PNG stills** import as a clip whose file duration is 18000 with
  `<alphatype>straight</alphatype>`.
- **Unused clips sit to the right of the cut**, on their own camera's track, starting 250 frames
  after the end, all switched off. First come the **recommended shots**: tag-scored action
  windows (about 4 s, natural sound, no crew or camera in frame, not static) cut from unused
  clips and labelled `REC section N: ...`. After them come the **whole unused clips**, so
  nothing is hidden. Say what the block is, because it looks like a second timeline.
- **Linked audio:** link elements carry `linkclipref`, `mediatype`, `trackindex`, `clipindex`.
- **Never let a standalone file overwrite the main one.** A montage-only file is its own
  script and its own output.

Handoff note (every time)

Tell the user, in plain words:
- what is on each track, and which tracks are switched off on purpose
- the block to the right of the cut (recommended shots first, then the whole unused clips),
  and to set the out point at the cut end before export
- how to relink: select the offline clips, Link Media, point at the camera folder; proxies
  attach afterwards via Attach Proxies; importing into a project that already has the
  footage does not duplicate it if they relink rather than re-import
- what is untested (cover windows chosen from tags, not from watching) and what to check first

Checklist before handing over

- [ ] The story was shared and the user answered before the build
- [ ] The opening follows the style, then the hook
- [ ] Every section has its key point and payoff; the sign-off is kept
- [ ] No cut lands mid-sentence; no crew voice or retake line in the cut
- [ ] Every second camera's mouth matches the voice in a spot check at three cuts
- [ ] Callouts exist for every item the style calls out, on their own track
- [ ] Validator is clean; stills and covers each checked in a frame
- [ ] Runtime reported against the style's target, with a cut-first list if over
- [ ] The handoff note was sent and ends with the user's questions

Gotchas

- Claude cannot hear. Sync and cover timing come from word timings and cross-correlation,
  never from "sounds right".
- Do not describe a shot you have not seen in a frame. One wrong description of what a shot
  showed cost a full rebuild.
- A short clip will not span a long cover. Check duration before assigning.
- `sed -i` fails on macOS; edit with Python or the Edit tool.